



Chapter 3: Decrease-and-conquer

Algorithms Analysis and Design (C03031)

Instructor: Assoc. Prof. Dr. Vo Thi Ngoc Chau

Email: chauvtn@hcmut.edu.vn



Course Learning Outcomes

1. Able to analyze the complexity of the algorithms (recursive or iterative) and estimate the efficiency of the algorithms.
2. Improve the ability to design algorithms in different areas *by decomposing a computing problem into advanced algorithm design strategies.*
3. Able to discuss NP-completeness.



References

- [1] Cormen, T. H., Leiserson, C. E, and Rivest, R. L., *Introduction to Algorithms*, The MIT Press, 2009.
- [2] Levitin, A., *Introduction to the Design and Analysis of Algorithms*, 3rd Edition, Pearson, 2012.
- [3] Sedgewick, R., *Algorithms in C++*, Addison-Wesley, 1998.
- [4] Weiss, M.A., *Data Structures and Algorithm Analysis in C*, The Benjamin/Cummings Publishing, 1993.



Course Outline

- ❑ Chapter 1. Fundamentals
- ❑ Chapter 2. Divide-and-Conquer Strategy
- ❑ **Chapter 3. Decrease-and-Conquer Strategy**
- ❑ Chapter 4. Transform-and-Conquer Strategy
- ❑ Chapter 5. Dynamic Programming and Greedy Strategies
- ❑ Chapter 6. Backtracking and Branch-and-Bound
- ❑ Chapter 7. NP-completeness
- ❑ Chapter 8. Approximation Algorithms



Chapter 3. Decrease-and-Conquer Strategy

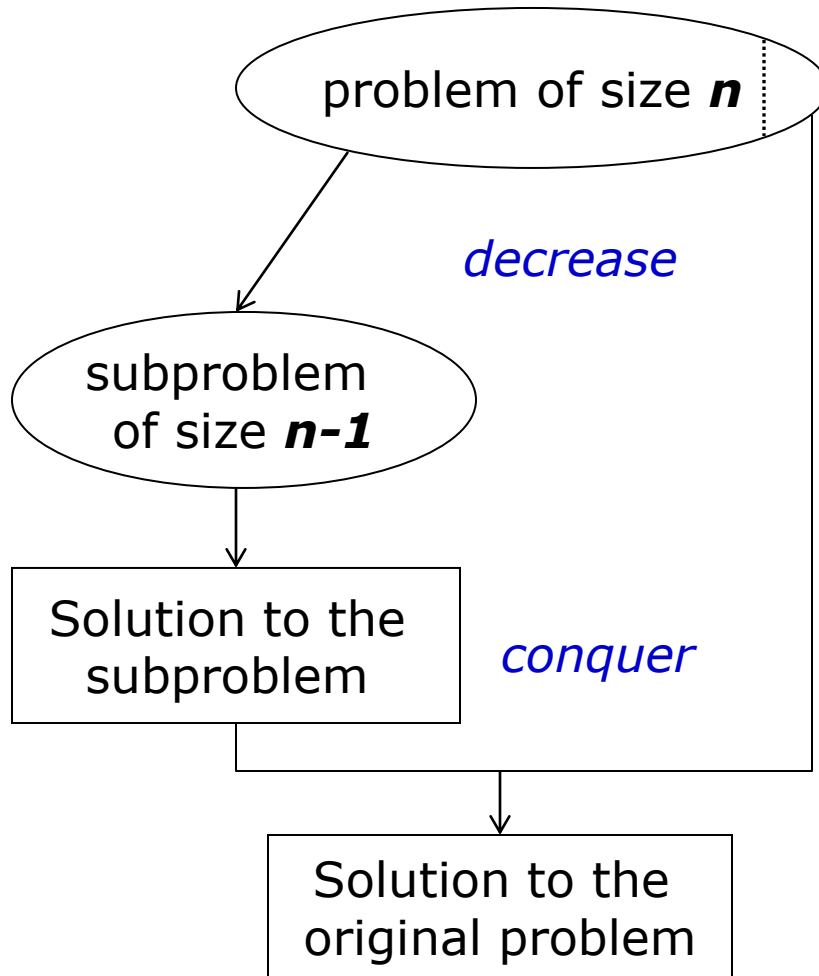
- ❑ 3.1. Decrease- and-conquer strategy
- ❑ 3.2. Insertion sort
- ❑ 3.3. Graph traversal algorithms
- ❑ 3.4. Topological sorting
- ❑ 3.5. Generating all permutations from a set



3.1. Decrease- and-conquer strategy

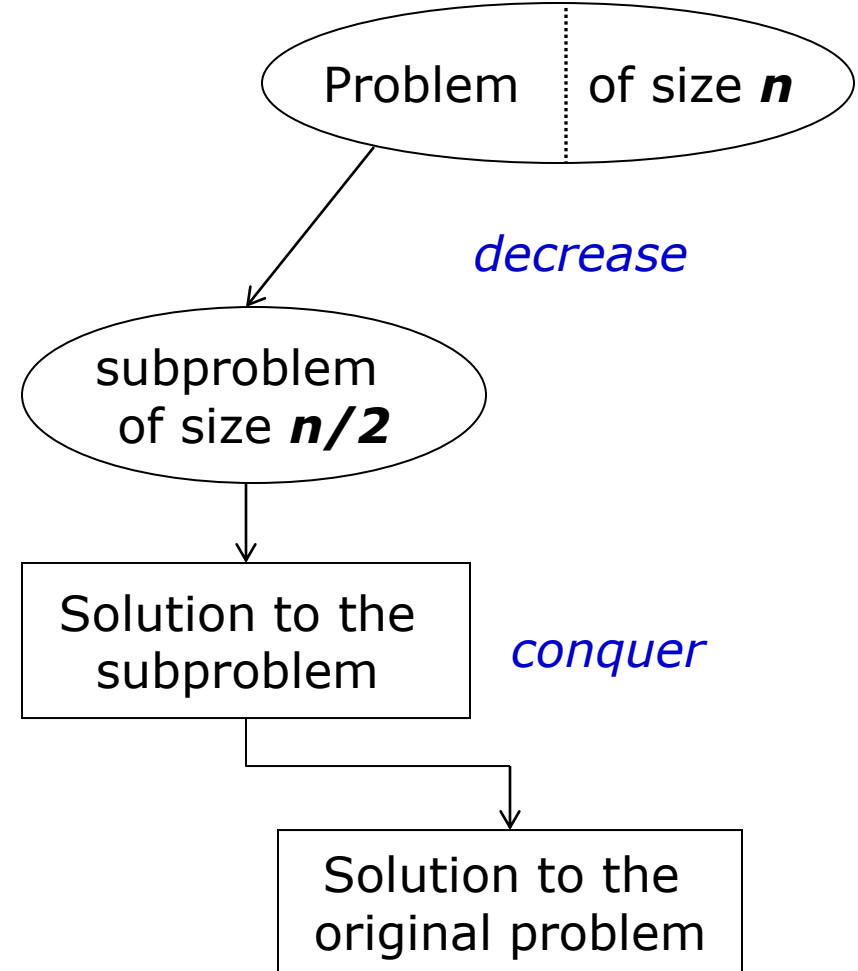
- ***Decrease-and-conquer*** strategy is based on exploiting the relationship between a solution to a given instance of a problem and a solution to a smaller instance of the same .
- Three major variations of decrease-and-conquer:
 - decrease by a *constant*
 - decrease by a *factor*
 - *variable-size* decrease

3.1. Decrease- and-conquer strategy



Decrease (by one)-and-conquer

Figure 5.1, [2], pp. 158.



Decrease (by half)-and-conquer

Figure 5.2, [2], pp. 159.



3.2. Insertion sort

□ Sort an array of n elements in ascending order. It is assumed that the smaller problem of sorting the array: $a[0..n-2]$ has already been solved. All we need is to find an appropriate position to insert $a[n-1]$ into the sorted subarray $a[0..n-2]$.

□ Two alternatives to insert $a[n-1]$ into the sorted array $a[0..n-2]$:

- i) Scan the sorted subarray from left to right until the first element *greater than or equal to* $a[n-1]$ is encountered and then insert $a[n-1]$ before that element.

1, 3, 7, 8, 10 ... 5 $\Rightarrow$ 1, 3, 5, 7, 8, 10

- ii) Scan the sorted subarray from right to left until the first element *smaller than or equal to* $a[n-1]$ is encountered and then insert $a[n-1]$ right after that element.

1, 3, 7, 8, 10 ... 5 $\Rightarrow$ 1, 3, 5, 7, 8, 10



3.2. Insertion sort

- Sort an array of n elements in ascending order. It is assumed that the smaller problem of sorting the array: $a[0..n-2]$ has already been solved. All we need is to find an appropriate position to insert $a[n-1]$ into the sorted subarray $a[0..n-2]$.
- Two alternatives to insert $a[n-1]$ into the sorted array $a[0..n-2]$:
 - i) Scan the sorted subarray from left to right until the first element *greater than or equal to* $a[n-1]$ is encountered and then insert $a[n-1]$ before that element.
 - ii) Scan the sorted subarray from right to left until the first element *smaller than or equal to* $a[n-1]$ is encountered and then insert $a[n-1]$ right after that element.

1, $\overleftarrow{3}$, 7, 8, 10 ... 5 $\Rightarrow$ 1, 3, 5, 7, 8, 10



3.2. Insertion sort

- Insertion sort – ascending order

8 8 9 2 1 0 0 7 6 11 4 9



0 0 1 2 4 6 7 8 8 9 9 11



3.2. Insertion sort

□ Insertion sort – ascending order

8	8	9	2	1	0	0	7	6	11	4	9
8											

sorted



3.2. Insertion sort

□ Insertion sort – ascending order

8	8	9	2	1	0	0	7	6	11	4	9
8											
8											

$8 \leq 8$



3.2. Insertion sort

□ Insertion sort – ascending order

8	8	9	2	1	0	0	7	6	11	4	9
8											
8	8										

$8 \leq 8$



3.2. Insertion sort

□ Insertion sort – ascending order

8	8	9	2	1	0	0	7	6	11	4	9
8											
8	8										
8	8										

$8 \leq 9$



3.2. Insertion sort

□ Insertion sort – ascending order

8	8	9	2	1	0	0	7	6	11	4	9
8											
8	8										
8	8	9									

$8 \leq 9$

3.2. Insertion sort

□ Insertion sort – ascending order

8	8	9	2	1	0	0	7	6	11	4	9
8											
8	8										
8	8	9									
8	8	9									

9 > 2

3.2. Insertion sort

□ Insertion sort – ascending order

8	8	9	2	1	0	0	7	6	11	4	9
8											
8	8										
8	8	9									
8	8	9	9								

9 > 2

3.2. Insertion sort

□ Insertion sort – ascending order

8	8	9	2	1	0	0	7	6	11	4	9
8											
8	8										
8	8	9									
8	8	8	9								



8 > 2

3.2. Insertion sort

□ Insertion sort – ascending order

8	8	9	2	1	0	0	7	6	11	4	9
8											
8	8										
8	8	9									
8	8	8	9								



8 > 2

3.2. Insertion sort

□ Insertion sort – ascending order

8	8	9	2	1	0	0	7	6	11	4	9
8											
8	8										
8	8	9									
2	8	8	9								

3.2. Insertion sort

□ Insertion sort – ascending order

8	8	9	2	1	0	0	7	6	11	4	9
8											
8	8										
8	8	9									
2	8	8	9								
2	8	8	9								

9 > 1

3.2. Insertion sort

□ Insertion sort – ascending order

8	8	9	2	1	0	0	7	6	11	4	9
8											
8	8										
8	8	9									
2	8	8	9								
2	8	8	9	9							

9 > 1

3.2. Insertion sort

□ Insertion sort – ascending order

8	8	9	2	1	0	0	7	6	11	4	9
8											
8	8										
8	8	9									
2	8	8	9								
2	8	8	8	9							

8 > 1

3.2. Insertion sort

□ Insertion sort – ascending order

8	8	9	2	1	0	0	7	6	11	4	9
8											
8	8										
8	8	9									
2	8	8	9								
2	8	8	8	9							



8 > 1

3.2. Insertion sort

□ Insertion sort – ascending order

8	8	9	2	1	0	0	7	6	11	4	9
8											
8	8										
8	8	9									
2	8	8	9								
2	2	8	8	9							



2 > 1

3.2. Insertion sort

□ Insertion sort – ascending order

8	8	9	2	1	0	0	7	6	11	4	9
8											
8	8										
8	8	9									
2	8	8	9								
1	2	8	8	9							

3.2. Insertion sort

□ Insertion sort – ascending order

8	8	9	2	1	0	0	7	6	11	4	9
8											
8	8										
8	8	9									
2	8	8	9								
1	2	8	8	9							
1	2	8	8	9							

9 > 0

3.2. Insertion sort

□ Insertion sort – ascending order

8	8	9	2	1	0	0	7	6	11	4	9
8											
8	8										
8	8	9									
2	8	8	9								
1	2	8	8	9							
1	2	8	8	9	9						



9 > 0

3.2. Insertion sort

□ Insertion sort – ascending order

8	8	9	2	1	0	0	7	6	11	4	9
8											
8	8										
8	8	9									
2	8	8	9								
1	2	8	8	9							
1	2	8	8	8	9						



8 > 0

3.2. Insertion sort

□ Insertion sort – ascending order

8	8	9	2	1	0	0	7	6	11	4	9
8											
8	8										
8	8	9									
2	8	8	9								
1	2	8	8	9							
1	2	8	8	8	9						



8 > 0

3.2. Insertion sort

□ Insertion sort – ascending order

8	8	9	2	1	0	0	7	6	11	4	9
8											
8	8										
8	8	9									
2	8	8	9								
1	2	8	8	9							
1	2	2	8	8	9						



2 > 0

3.2. Insertion sort

□ Insertion sort – ascending order

8	8	9	2	1	0	0	7	6	11	4	9
8											
8	8										
8	8	9									
2	8	8	9								
1	2	8	8	9							
1	1	2	8	8	9						

1 > 0

3.2. Insertion sort

□ Insertion sort – ascending order

8	8	9	2	1	0	0	7	6	11	4	9
8											
8	8										
8	8	9									
2	8	8	9								
1	2	8	8	9							
0	1	2	8	8	9						

3.2. Insertion sort

□ Insertion sort – ascending order

8	8	9	2	1	0	0	7	6	11	4	9
8											
8	8										
8	8	9									
2	8	8	9								
1	2	8	8	9							
0	1	2	8	8	9						
0	1	2	8	8	9	9					

9 > 0



3.2. Insertion sort

□ Insertion sort – ascending order

8	8	9	2	1	0	0	7	6	11	4	9
8											
8	8										
8	8	9									
2	8	8	9								
1	2	8	8	9							
0	1	2	8	8	9						
0	1	2	8	8	8	9					

8 > 0

3.2. Insertion sort

□ Insertion sort – ascending order

8	8	9	2	1	0	0	7	6	11	4	9
8											
8	8										
8	8	9									
2	8	8	9								
1	2	8	8	9							
0	1	2	8	8	9						
0	1	2	8	8	8	9					



8 > 0

3.2. Insertion sort

□ Insertion sort – ascending order

8	8	9	2	1	0	0	7	6	11	4	9
8											
8	8										
8	8	9									
2	8	8	9								
1	2	8	8	9							
0	1	2	8	8	9						
0	1	2	2	8	8	9					



2 > 0

3.2. Insertion sort

□ Insertion sort – ascending order

8	8	9	2	1	0	0	7	6	11	4	9
8											
8	8										
8	8	9									
2	8	8	9								
1	2	8	8	9							
0	1	2	8	8	9						
0	1	1	2	8	8	9					

1 > 0

3.2. Insertion sort

□ Insertion sort – ascending order

8	8	9	2	1	0	0	7	6	11	4	9
8											
8	8										
8	8	9									
2	8	8	9								
1	2	8	8	9							
0	1	2	8	8	9						
0	0	1	2	8	8	9					

$0 \leq 0$

3.2. Insertion sort

□ Insertion sort – ascending order

8	8	9	2	1	0	0	7	6	11	4	9
8											
8	8										
8	8	9									
2	8	8	9								
1	2	8	8	9							
0	1	2	8	8	9						
0	0	1	2	8	8	9					
0	0	1	2	8	8	9	9				

9 > 7

3.2. Insertion sort

□ Insertion sort – ascending order

8	8	9	2	1	0	0	7	6	11	4	9
8											
8	8										
8	8	9									
2	8	8	9								
1	2	8	8	9							
0	1	2	8	8	9						
0	0	1	2	8	8	9					
0	0	1	2	8	8	8	9				

8 > 7

3.2. Insertion sort

□ Insertion sort – ascending order

8	8	9	2	1	0	0	7	6	11	4	9
8											
8	8										
8	8	9									
2	8	8	9								
1	2	8	8	9							
0	1	2	8	8	9						
0	0	1	2	8	8	9					
0	0	1	2	8	8	8	9				

8 > 7

3.2. Insertion sort

□ Insertion sort – ascending order

8	8	9	2	1	0	0	7	6	11	4	9
8											
8	8										
8	8	9									
2	8	8	9								
1	2	8	8	9							
0	1	2	8	8	9						
0	0	1	2	8	8	9					
0	0	1	2	7	8	8	9				

$2 \leq 7$

3.2. Insertion sort

□ Insertion sort – ascending order

8	8	9	2	1	0	0	7	6	11	4	9
8											
8	8										
8	8	9									
2	8	8	9								
1	2	8	8	9							
0	1	2	8	8	9						
0	0	1	2	8	8	9					
0	0	1	2	7	8	8	9				
0	0	1	2	6	7	8	8	9			

3.2. Insertion sort

□ Insertion sort – ascending order

8	8	9	2	1	0	0	7	6	11	4	9
8											
8	8										
8	8	9									
2	8	8	9								
1	2	8	8	9							
0	1	2	8	8	9						
0	0	1	2	8	8	9					
0	0	1	2	7	8	8	9				
0	0	1	2	6	7	8	8	9			
0	0	1	2	6	7	8	8	9	11		

3.2. Insertion sort

□ Insertion sort – ascending order

8	8	9	2	1	0	0	7	6	11	4	9
8											
8	8										
8	8	9									
2	8	8	9								
1	2	8	8	9							
0	1	2	8	8	9						
0	0	1	2	8	8	9					
0	0	1	2	7	8	8	9				
0	0	1	2	6	7	8	8	9			
0	0	1	2	6	7	8	8	9	11		
0	0	1	2	4	6	7	8	8	9	11	

3.2. Insertion sort

□ Insertion sort – ascending order

8	8	9	2	1	0	0	7	6	11	4	9
8											
8	8										
8	8	9									
2	8	8	9								
1	2	8	8	9							
0	1	2	8	8	9						
0	0	1	2	8	8	9					
0	0	1	2	7	8	8	9				
0	0	1	2	6	7	8	8	9			
0	0	1	2	6	7	8	8	9	11		
0	0	1	2	4	6	7	8	8	9	11	
0	0	1	2	4	6	7	8	8	9	9	11

3.2. Insertion sort

```
procedure insertion;  
var i; j; v:integer;  
begin  
    for i:=2 to N do  
        begin  
            v:=a[i]; j:= i;  
            while a[j-1]> v do  
                begin  
                    a[j] := a[j-1]; //shift  
                    j:= j-1;  
                end;  
            a[j]:=v;  
        end;  
    end;
```



3.2. Insertion sort

- Time complexity for sorting n elements

- Abstract operation: *comparison*

- Best case:

$$C_n = n-1 = O(n)$$

- Average case:

$$C_n = n(n-1)/4 = O(n^2)$$

- Worst case:

$$C_n = n(n-1)/2 = O(n^2)$$

3.2. Insertion sort

- Time complexity for sorting n elements
 - Abstract operation: *comparison*
 - Best case:

$$C_n = n-1 = O(n)$$

	8	8	9
0 cmps	8		
1 cmps	8	8	
1 cmps	8	8	9

Best case when n elements are in order.
 Each of the 2nd to n -th elements needs 1 comparison to identify its position in a sorted array.

$$C_n = n-1 = O(n)$$



3.2. Insertion sort

- Time complexity for sorting n elements
 - Abstract operation: *comparison*
 - Best case:

$$C_n = n-1 = O(n)$$

	8	8	9
0 cmps	8		
1 cmps	8	8	
1 cmps	8	8	9

Best case when n elements are in order.

Each of the 2nd to n -th elements needs 1 comparison to identify its position in a sorted array.

$$C_n = n-1 = O(n)$$



3.2. Insertion sort

- Time complexity for sorting n elements
 - Abstract operation: *comparison*
 - Average case:

$$C_n = n(n-1)/4 = O(n^2)$$

$a[i]$ can be inserted at any position from 1 to i with the same probability $1/i$.

At: i : 1 cmps
 $i-1$: 2 cmps
 ...
 1: $i-1$ cmps

$$C_n = \sum_{i=2}^n \frac{1}{i} (1 + 2 + \dots + (i-1))$$

0 cmps
 1 cmps
 1 cmps
 3 cmps
 4 cmps
 5 cmps
 6 cmps
 4 cmps

8	8	9	2	1	0	0	7
8							
8	8						
8	8	9					
2	8	8	9				
1	2	8	8	9			
0	1	2	8	8	9		
0	0	1	2	8	8	9	
0	0	1	2	7	8	8	9



3.2. Insertion sort

- Time complexity for sorting n elements
 - Abstract operation: *comparison*
 - Average case:

$$C_n = n(n-1)/4 = O(n^2)$$

$a[i]$ can be inserted at any position from 1 to i with the same probability $1/i$.

$$C_n = \sum_{i=2}^n \frac{1}{i} (1 + 2 + \dots + (i-1))$$

$$C_n = \sum_{i=2}^n \frac{i-1}{2} = \frac{n(n-1)}{2 * 2} = O(n^2)$$

{ 0 cmps
 1 cmps
 1 cmps
 3 cmps
 4 cmps
 5 cmps
 6 cmps
 4 cmps

8	8	9	2	1	0	0	7
8							
8	8						
8	8	9					
2	8	8	9				
1	2	8	8	9			
0	1	2	8	8	9		
0	0	1	2	8	8	9	
0	0	1	2	7	8	8	9



3.2. Insertion sort

- Time complexity for sorting n elements
 - Abstract operation: *comparison*
 - Average case:

$$C_n = n(n-1)/4 = O(n^2)$$

$a[i]$ can be inserted at any position from 1 to i with the same probability $1/i$.

$$C_n = \sum_{i=2}^n \frac{1}{i} (1 + 2 + \dots + (i-1))$$

$$C_n = \sum_{i=2}^n \frac{i-1}{2} = \frac{n(n-1)}{2 * 2} = O(n^2)$$

{ 0 cmps
 1 cmps
 1 cmps
 3 cmps
 4 cmps
 5 cmps
 6 cmps
 4 cmps

8	8	9	2	1	0	0	7
8							
8	8						
8	8	9					
2	8	8	9				
1	2	8	8	9			
0	1	2	8	8	9		
0	0	1	2	8	8	9	
0	0	1	2	7	8	8	9

3.2. Insertion sort

- Time complexity for sorting n elements
 - Abstract operation: *comparison*
 - Worst case:

$$C_n = n(n-1)/2 = O(n^2)$$

	9	2	1	0
0 cmps	9			
1 cmps	2	9		
2 cmps	1	2	9	
3 cmps	0	1	2	9

Worst case when n elements are in reverse order.

Each of the 2nd to n -th elements needs $i-1$ comparisons to identify its position at the first position in the array.

$$C_n = \sum_{i=2}^n (i-1) = \frac{n(n-1)}{2} = O(n^2)$$

3.2. Insertion sort

- Time complexity for sorting n elements
 - Abstract operation: *comparison*
 - Worst case:

$$C_n = n(n-1)/2 = O(n^2)$$

	9	2	1	0
0 cmps	9			
1 cmps	2	9		
2 cmps	1	2	9	
3 cmps	0	1	2	9

Worst case when n elements are in reverse order.

Each of the 2nd to n -th elements needs $i-1$ comparisons to identify its position at the first position in the array.

$$C_n = \sum_{i=2}^n (i-1) = \frac{n(n-1)}{2} = O(n^2)$$



3.2. Insertion sort

□ Your turn:

3.2.1. Trace by hand Insertion sort

(3.2.1.1) 23, 7, 92, 6, 12, 24, 40, 44, 20, 21

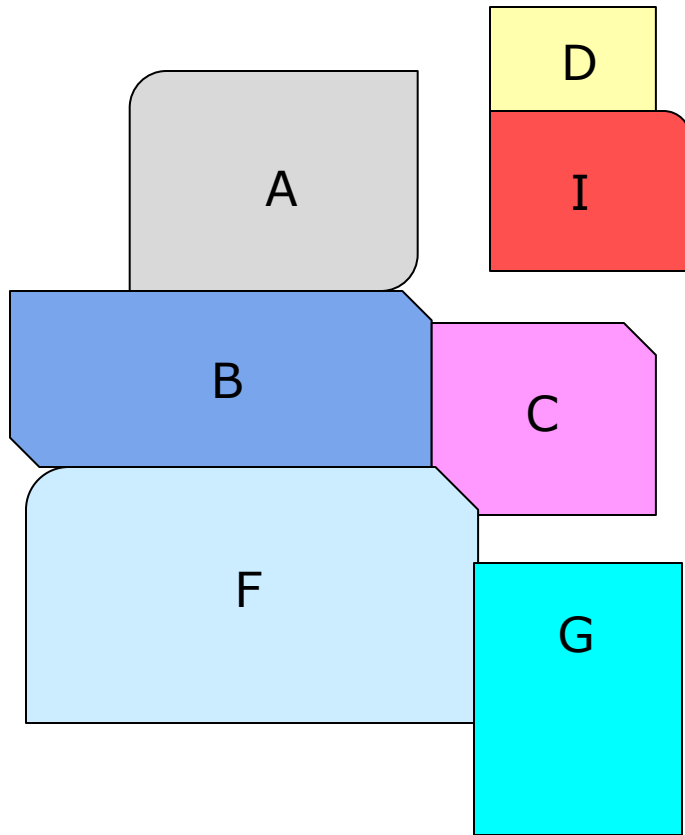
(3.2.1.2) 44, 30, 50, 22, 60, 55, 77, 55

(3.2.1.3) 2, 4, 6, 8, 10, 10

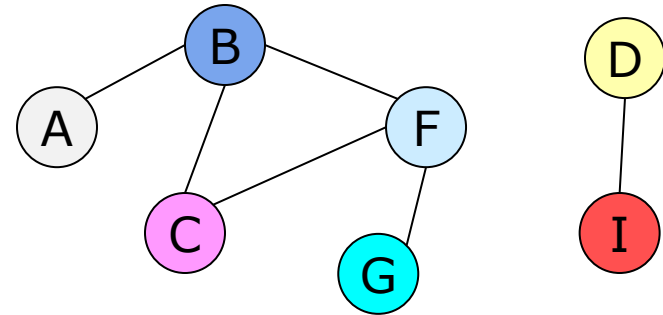
(3.2.1.4) 13, 11, 7, 7, 3, 2, 1

3.2.2. What is Insertion sort's complexity for shifts?

3.3. Graph traversal algorithms



A map
in our **real** world



Graph modeling
to represent direct connections

A: -B
B: -A-C-F
C: -B-F
D: -I
F: -B-C-G
G: -F
I: -D

Adjacency lists/matrix

	A	B	C	D	F	G	I
A	1	1	0	0	0	0	0
B	1	1	1	0	1	0	0
C	0	1	1	0	1	0	0
D	0	0	0	1	0	0	1
F	0	1	1	0	1	1	0
G	0	0	0	0	1	1	0
I	0	0	0	1	0	0	1

Graph representation
in the **computer's** world

3.3. Graph traversal algorithms

- Time complexity for checking if an edge between vertex u and vertex v exists in a graph of V vertices and E edges
 - Abstract operation: *comparison*
 - Adjacency list: $O(|V|)$ in the worst case
 - Adjacency matrix: $O(1)$ in all the cases

Adjacency lists

A: -B
B: -A-C-F
C: -B-F
D: -I
F: -B-C-G
G: -F
I: -D

	A	B	C	D	F	G	I
A	1	1	0	0	0	0	0
B	1	1	1	0	1	0	0
C	0	1	1	0	1	0	0
D	0	0	0	1	0	0	1
F	0	1	1	0	1	1	0
G	0	0	0	0	1	1	0
I	0	0	0	1	0	0	1

Adjacency matrix



3.3. Graph traversal algorithms

- ❑ ***Graph traversal***: to “visit” every vertex and check every edge in the graph systematically.
- ❑ Two main methods for graph traversal:
 - *depth-first search* (DFS)
 - ❑ Recursive visits
 - ❑ Stack-based visits
 - *breadth-first search* (BFS)
 - ❑ Queue-based visits



3.3. Graph traversal algorithms

```
type link = ↑node
node = record v: integer; next: link end;
```

```
procedure list-dfs;
```

```
var id, k: integer;
```

```
val: array[1..maxV] of integer;
```

```
procedure visit (k: integer);
```

```
var t: link;
```

```
begin
```

```
id:= id + 1; val[k]:= id; /* change the status of k to “visited” */
```

```
t:= adj[k]; /* find the neighbors of the vertex k */
```

```
while t <> z do /*z = empty*/
```

```
begin
```

```
if val[t↑.v] = 0 then visit(t↑.v);
```

```
t:= t↑.next
```

```
end
```

```
end;
```

```
begin
```

```
id:= 0;
```

```
for k:= 1 to V do val[k]:= 0; /*initialize the status of all vertices */
```

```
for k:= 1 to V do
```

```
if val[k] = 0 then visit(k)
```

```
end;
```

□ **Depth-first search**
with **recursive** visits

$val[1..V]$: status of V vertices

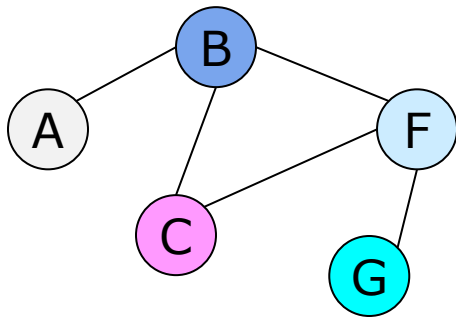
$val[k] = 0$ if vertex k has not yet been visited

$val[k] \neq 0$ if vertex k has already been visited

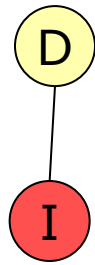
$val[k] := j$ implies that vertex k is the j -th visited vertex

3.3. Graph traversal algorithms

□ **Depth-first search** with **recursive** visits



Graph

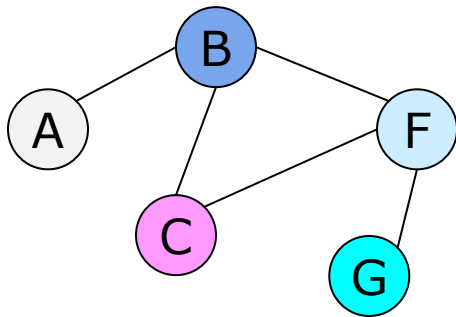


Starting at **F**: ???

A: -B
B: -A-C-F
C: -B-F
D: -I
F: -B-C-G
G: -F
I: -D

3.3. Graph traversal algorithms

□ *Depth-first search* with *recursive* visits



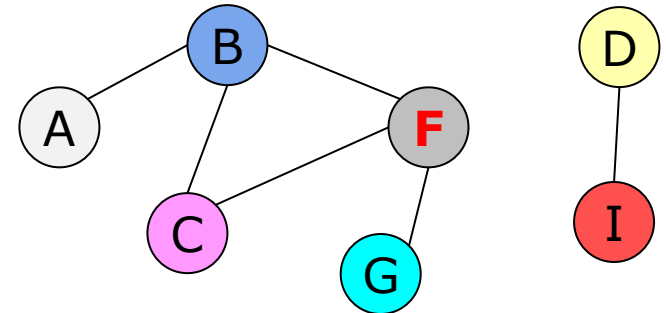
Graph

A: -B
B: -A-C-F
C: -B-F
D: -I
F: -B-C-G
G: -F
I: -D

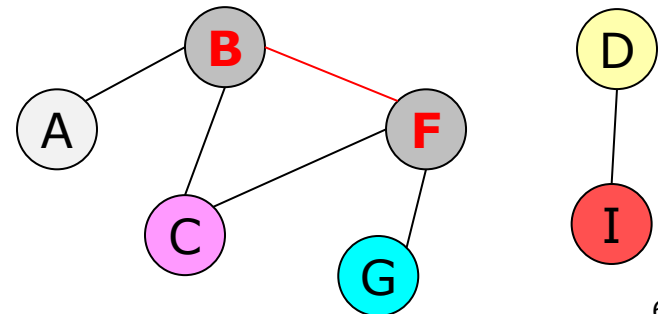
Adjacency lists

Starting at **F**:

Visit F, check F's list: -B-C-G

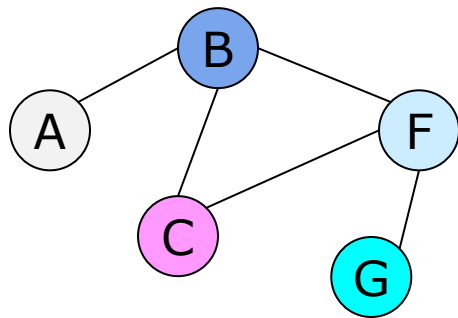


Visit B, check B's list: -A-C-**F**



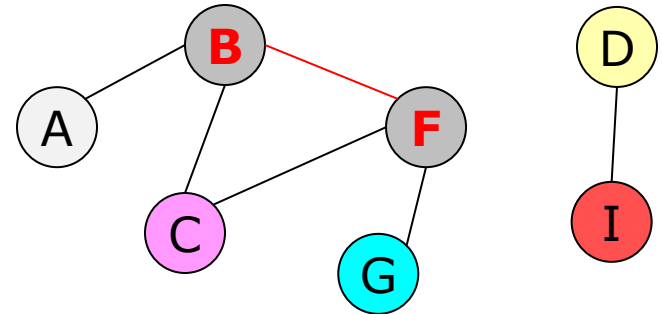
3.3. Graph traversal algorithms

□ *Depth-first search* with *recursive* visits

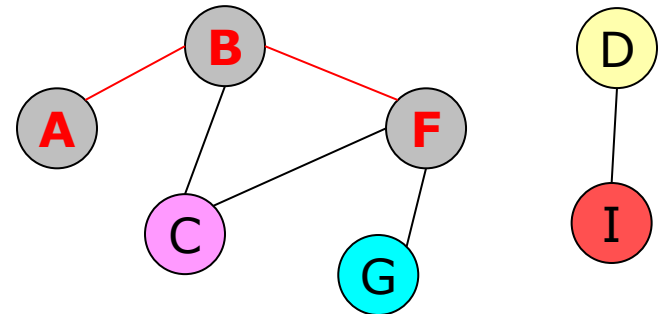


Graph

Visit B, check B's list: -A-C-**F**



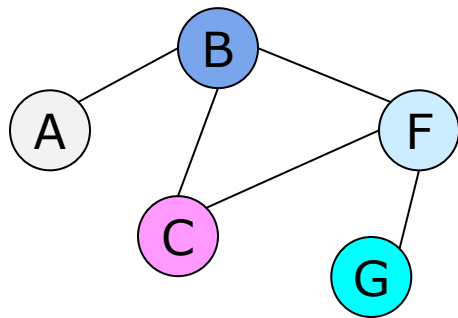
Visit A, check A's list: -**B**, check B's list: -**A**-C-**F**



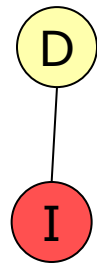
A: -B
B: -A-C-F
C: -B-F
D: -I
F: -B-C-G
G: -F
I: -D

3.3. Graph traversal algorithms

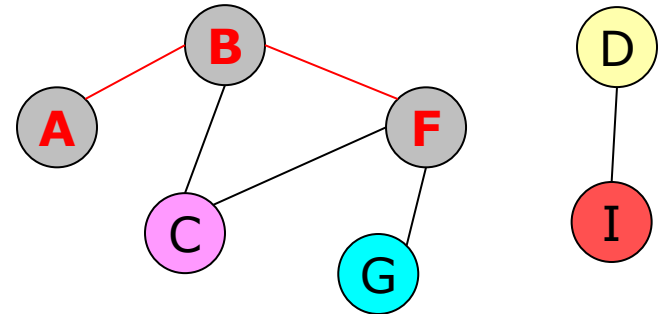
□ *Depth-first search* with *recursive* visits



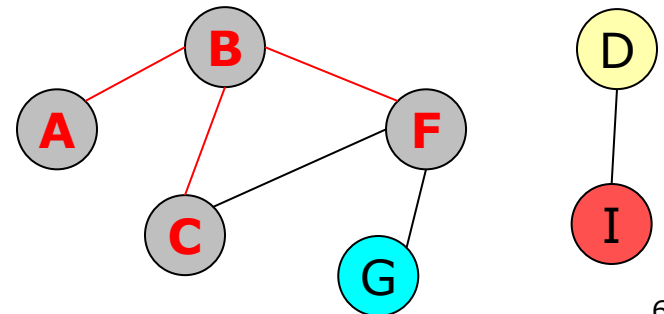
Graph



Visit A, check A's list: -**B**, check B's list: -**A-C-F**



Visit C, check C's list: -**B-F**, check B's list: -**A-C-F**,
check F's list: -**B-C-G**



A: -B
B: -A-C-F
C: -B-F
D: -I
F: -B-C-G
G: -F
I: -D

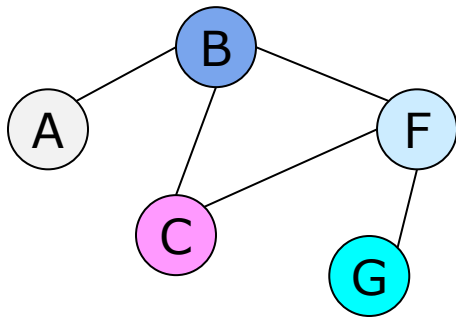
Adjacency lists

3.3. Graph traversal algorithms

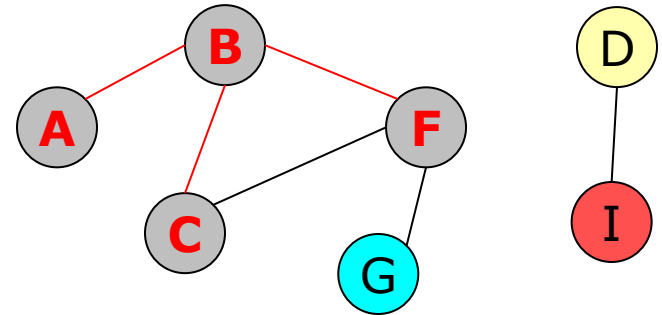
□ **Depth-first search** with **recursive** visits

Visit C, check C's list: -**B-F**, check B's list: -**A-C-F**,

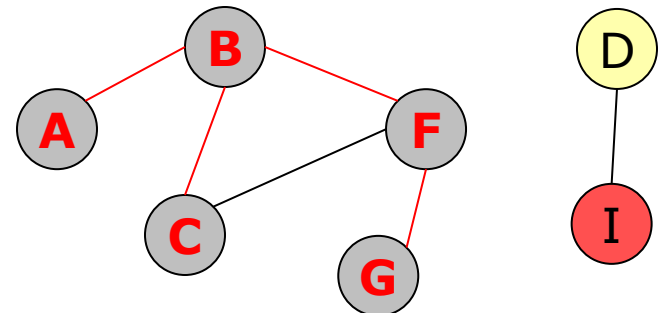
check F's list: -**B-C-G**



Graph



Visit G, check G's list: -**F**

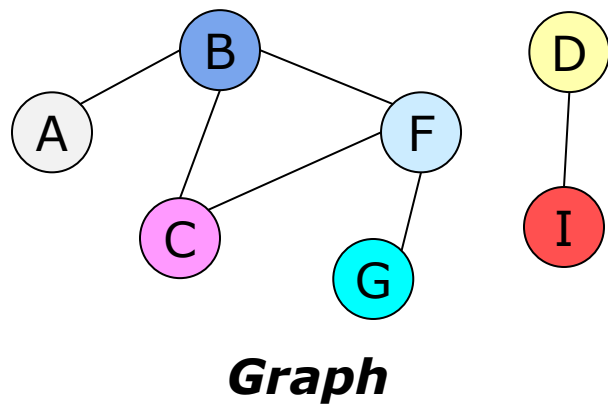


A: -B
B: -A-C-F
C: -B-F
D: -I
F: -B-C-G
G: -F
I: -D

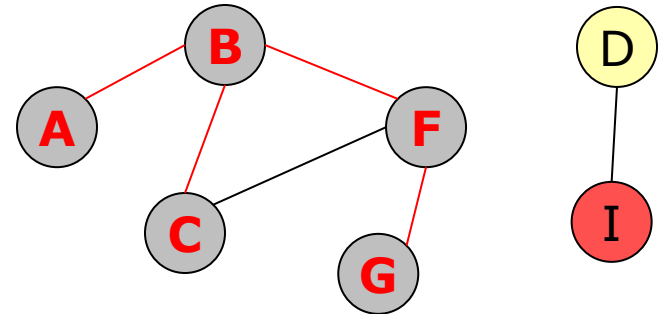
Adjacency lists

3.3. Graph traversal algorithms

□ **Depth-first search** with **recursive** visits

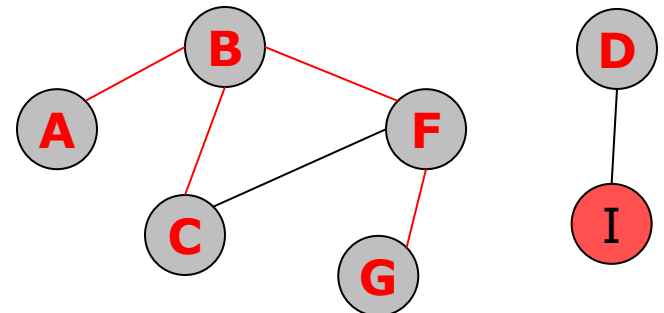


Visit G, check G's list: -**F**



A: -B
B: -A-C-F
C: -B-F
D: -I
F: -B-C-G
G: -F
I: -D

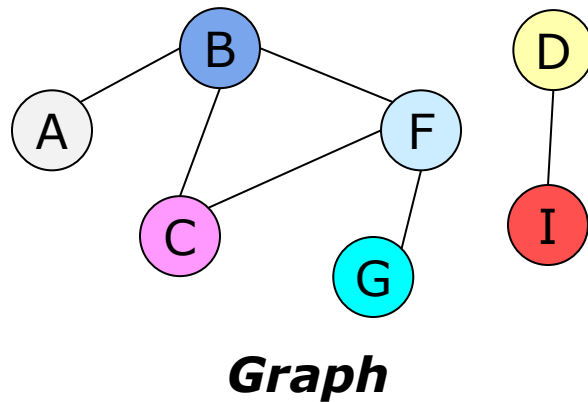
Visit D, check D's list: -**I**



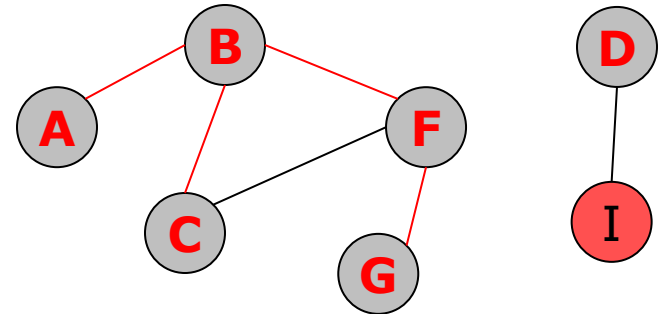
Adjacency lists

3.3. Graph traversal algorithms

□ **Depth-first search** with **recursive** visits

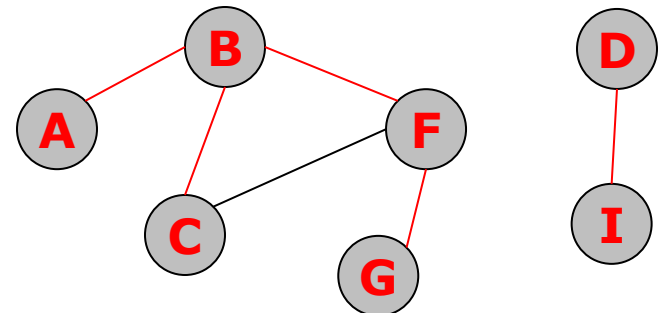


Visit D, check D's list: -I



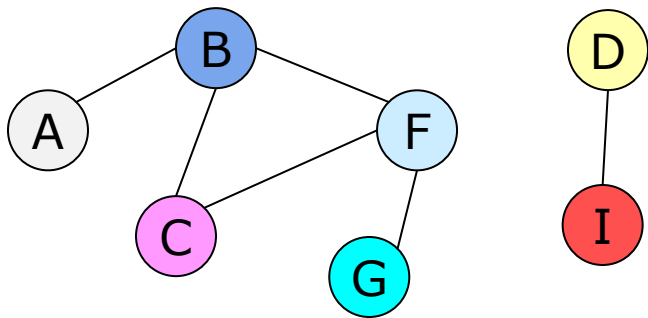
A: -B
B: -A-C-F
C: -B-F
D: -I
F: -B-C-G
G: -F
I: -D

Visit I, check I's list: -**D**



3.3. Graph traversal algorithms

□ **Depth-first search** with **recursive** visits



Graph

A: -B
B: -A-C-F
C: -B-F
D: -I
F: -B-C-G
G: -F
I: -D

Starting at **F**:

- Visit **F**, check F's list
- Visit **B**, check B's list
- Visit **A**, check A's list, B's list
- Visit **C**, check C's list, B's list, F's list
- Visit **G**, check G's list
- Visit **D**, check D's list
- Visit **I**, check I's list



3.3. Graph traversal algorithms

procedure *dfs*;

 procedure *visit*(*n*:vertex);

 begin

 add *n* to the ready stack;

while the ready stack is not empty **do**

 get a vertex from the stack, process it,

 and add any neighbor vertex that has not been processed to the stack.

 if a vertex has already appeared in the stack, there is no need to push it to the stack.

end;

begin

 Initialize status;

for each vertex, say *n*, in the graph **do**

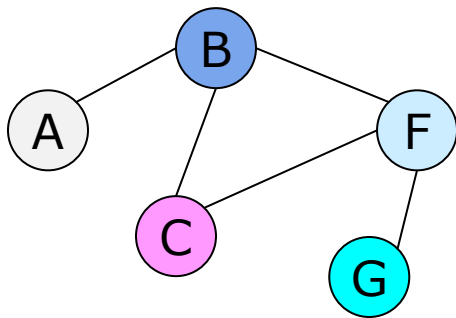
if the status of *n* is “not yet visited” **then** *visit*(*n*)

end;

□ ***Depth-first search***
with ***stack***-based visits

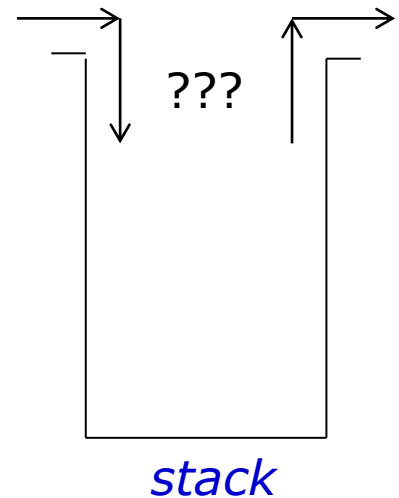
3.3. Graph traversal algorithms

□ **Depth-first search** with **stack**-based visits



Graph

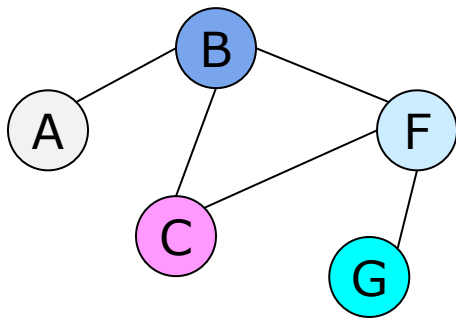
Starting at **F**: ???



A: -B
B: -A-C-F
C: -B-F
D: -I
F: -B-C-G
G: -F
I: -D

3.3. Graph traversal algorithms

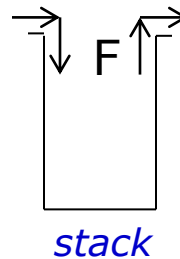
□ **Depth-first search** with **stack**-based visits



Graph

Starting at **F**:

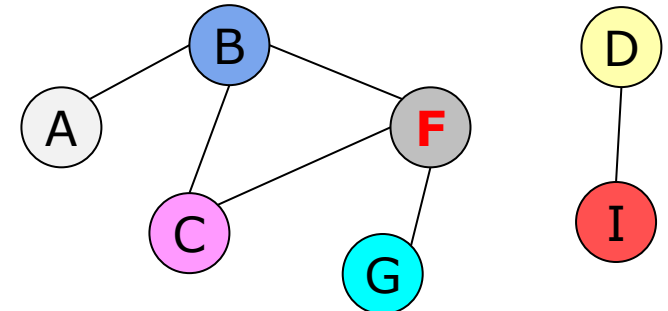
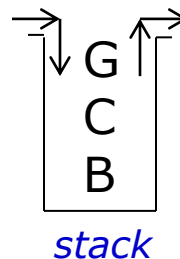
Push F into stack



A: -B
B: -A-C-F
C: -B-F
D: -I
F: -B-C-G
G: -F
I: -D

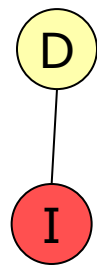
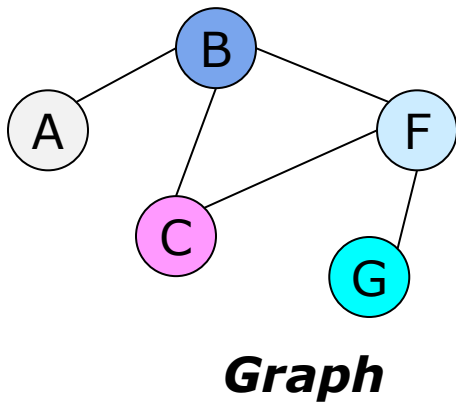
Adjacency lists

Visit F, Check F's list, Push B, C, G into stack

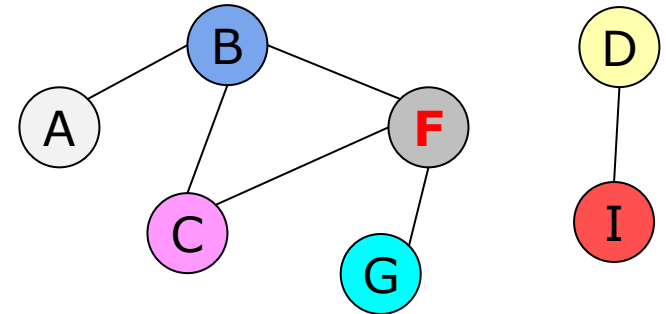
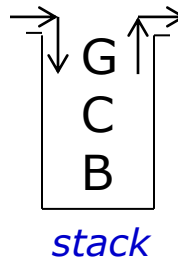


3.3. Graph traversal algorithms

□ **Depth-first search** with **stack**-based visits

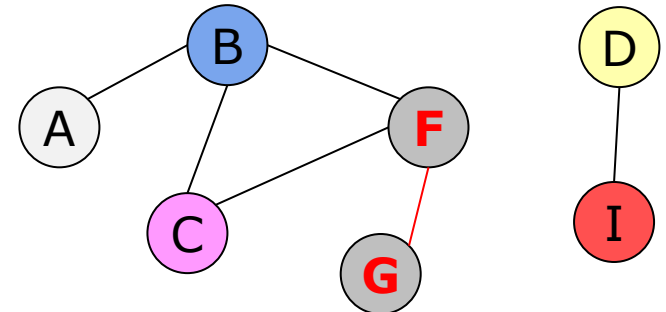
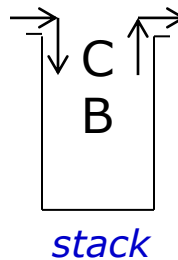


Visit F, Check F's list, Push B, C, G into stack



A: -B
B: -A-C-F
C: -B-F
D: -I
F: -B-C-G
G: -F
I: -D

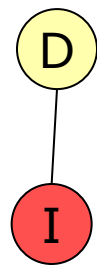
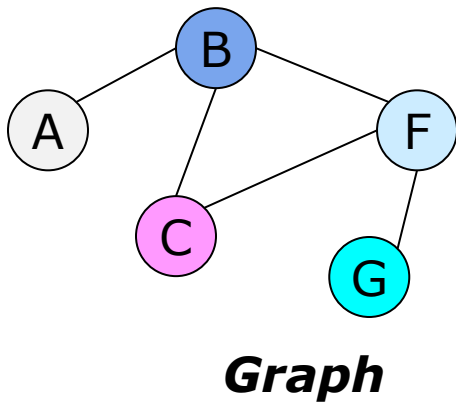
Visit G, Check G's list



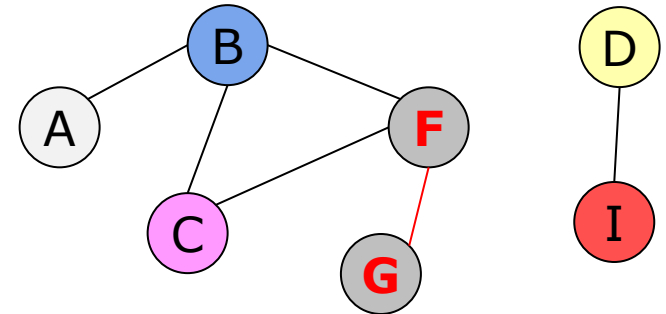
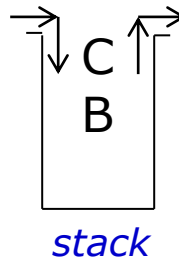
Adjacency lists

3.3. Graph traversal algorithms

□ **Depth-first search** with **stack**-based visits

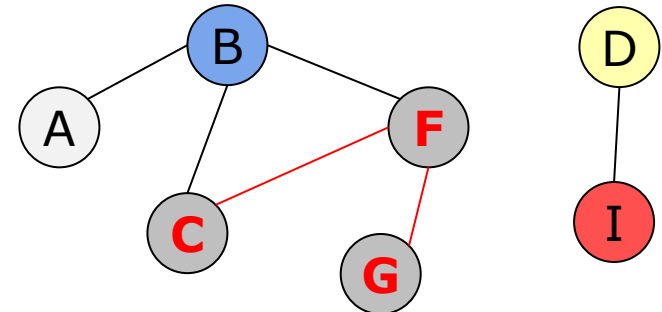
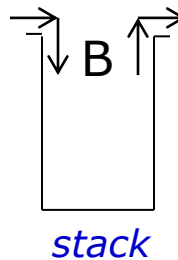


Visit G, Check G's list



A: -B
B: -A-C-F
C: -B-F
D: -I
F: -B-C-G
G: -F
I: -D

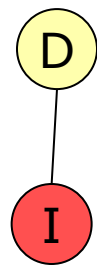
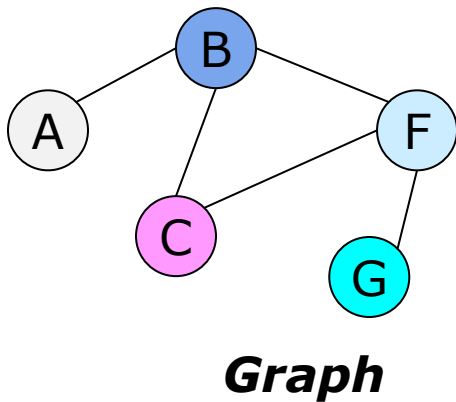
Visit C, Check C's list



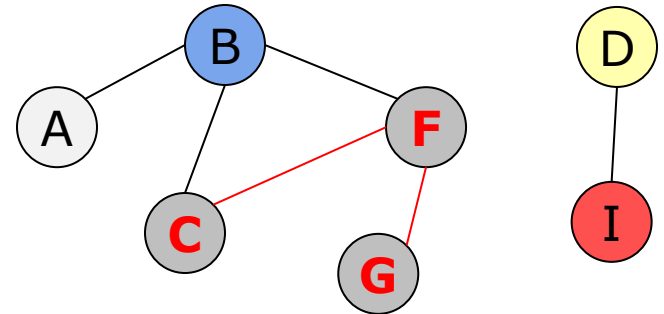
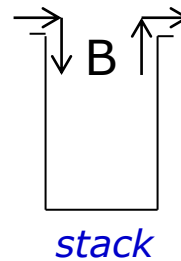
Adjacency lists

3.3. Graph traversal algorithms

□ **Depth-first search** with **stack**-based visits



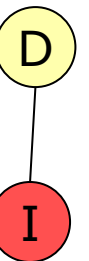
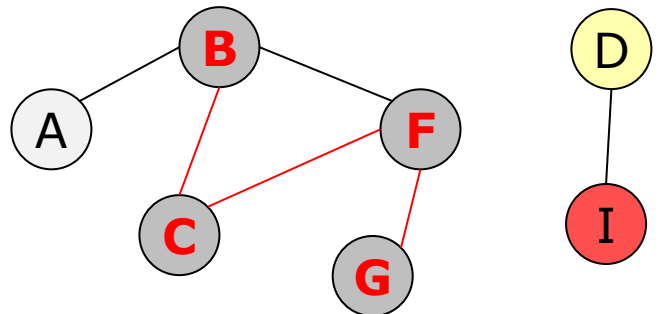
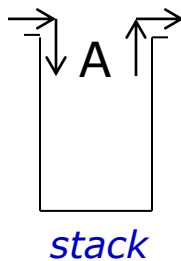
Visit C, Check C's list



A: -B
B: -A-C-F
C: -B-F
D: -I
F: -B-C-G
G: -F
I: -D

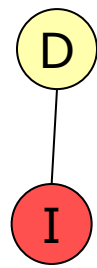
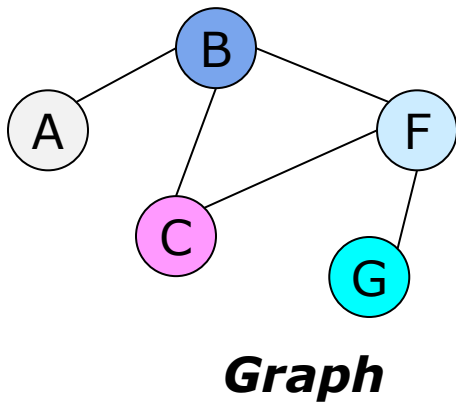
Adjacency lists

Visit B, Check B's list, Push A into stack

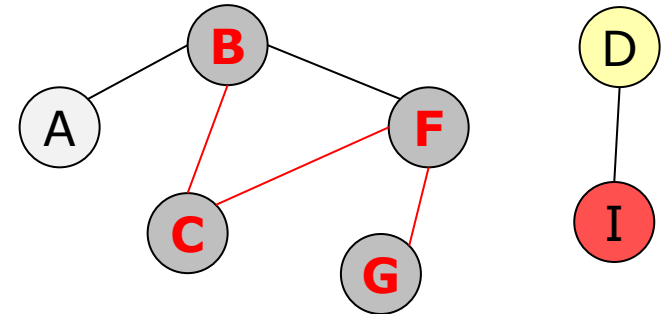
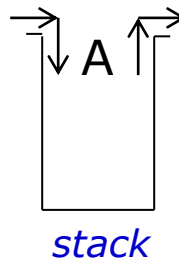


3.3. Graph traversal algorithms

□ **Depth-first search** with **stack**-based visits

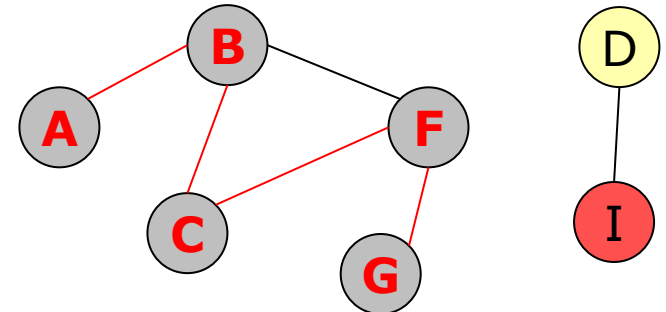
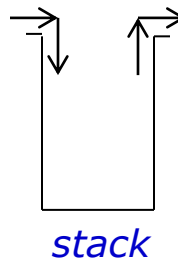


Visit B, Check B's list, Push A into stack



A: -B
B: -A-C-F
C: -B-F
D: -I
F: -B-C-G
G: -F
I: -D

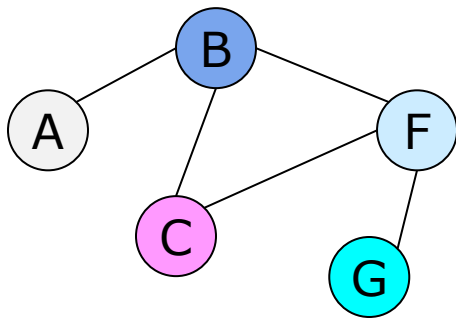
Visit A, Check A's list



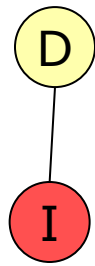
Adjacency lists

3.3. Graph traversal algorithms

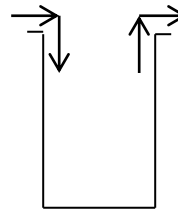
□ **Depth-first search** with **stack**-based visits



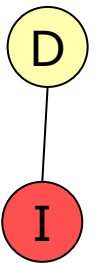
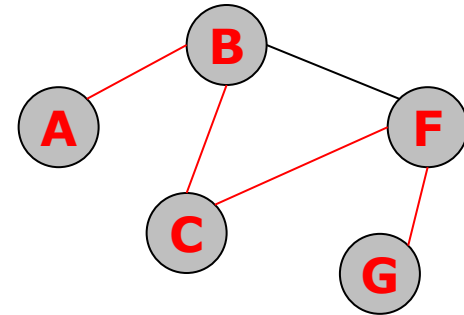
Graph



Visit A, Check A's list

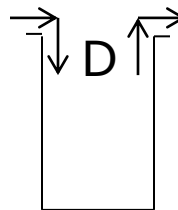


stack

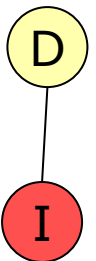
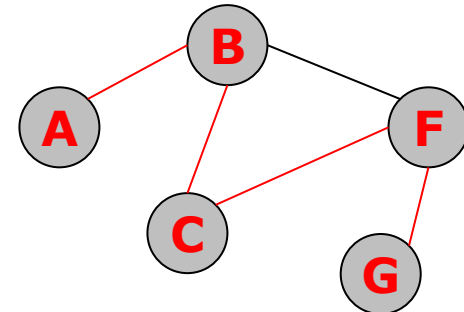


A: -B
B: -A-C-F
C: -B-F
D: -I
F: -B-C-G
G: -F
I: -D

Push D into stack

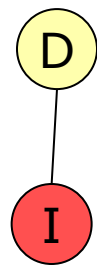
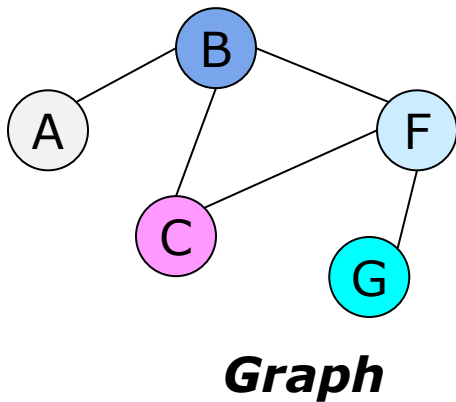


stack

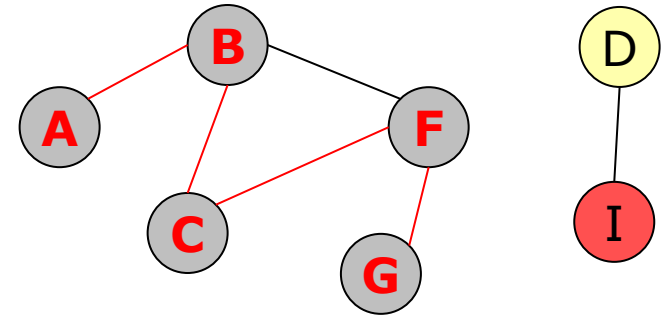
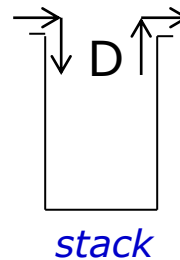


3.3. Graph traversal algorithms

□ **Depth-first search** with **stack**-based visits

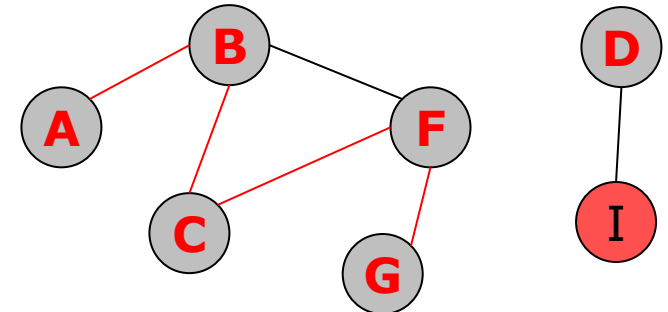
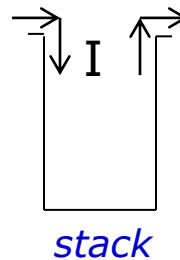


Push D into stack



A: -B
B: -A-C-F
C: -B-F
D: -I
F: -B-C-G
G: -F
I: -D

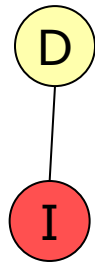
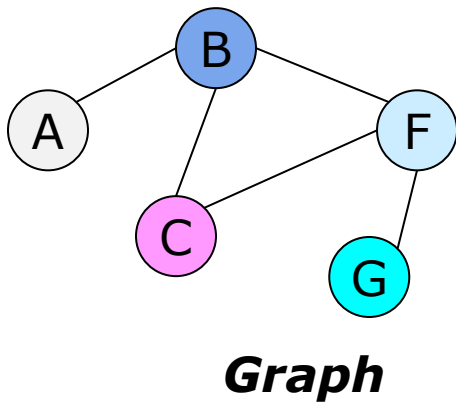
Visit D, Check D's list, Push I into stack



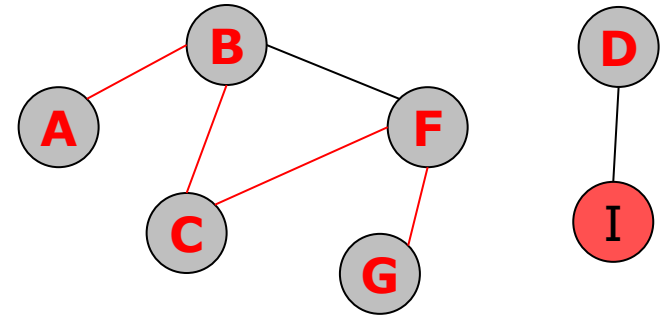
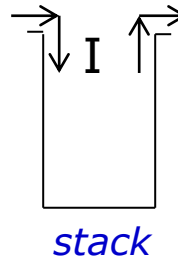
Adjacency lists

3.3. Graph traversal algorithms

□ **Depth-first search** with **stack**-based visits

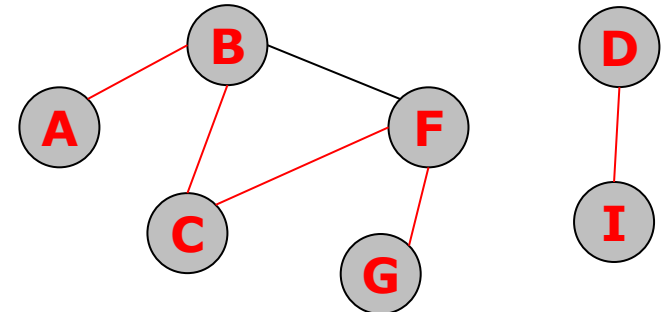
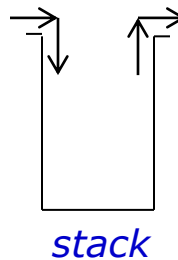


Visit D, Check D's list, Push I into stack



A: -B
B: -A-C-F
C: -B-F
D: -I
F: -B-C-G
G: -F
I: -D

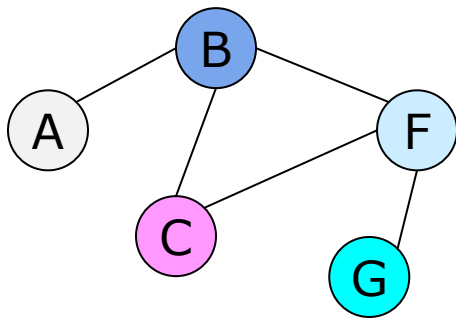
Visit I, Check I's list



Adjacency lists

3.3. Graph traversal algorithms

□ **Depth-first search** with **stack**-based visits



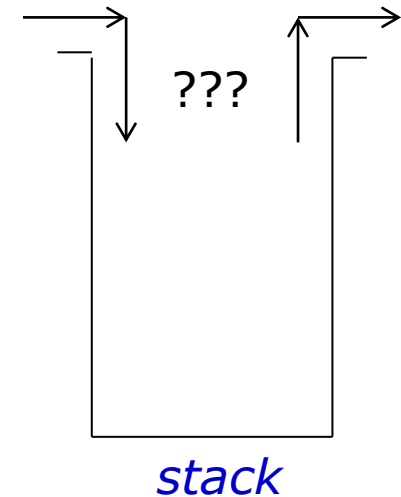
Graph

A: -B
B: -A-C-F
C: -B-F
D: -I
F: -B-C-G
G: -F
I: -D

Adjacency lists

Starting at **F**:

- Visit **F**, Push B, C, G
- Visit **G**
- Visit **C**
- Visit **B**, Push A
- Visit **A**
- Visit **D**, Push I
- Visit **I**





3.3. Graph traversal algorithms

procedure **bfs**;

 procedure **visit**(n : vertex);

 begin

 add n to the ready queue;

while the ready queue is not empty **do**

 get a vertex from the queue, process it, and add any
 neighbor vertex that has not been processed to the queue
 and change their status to ready.

 end;

begin

 Initialize status;

for each vertex, say n , in the graph

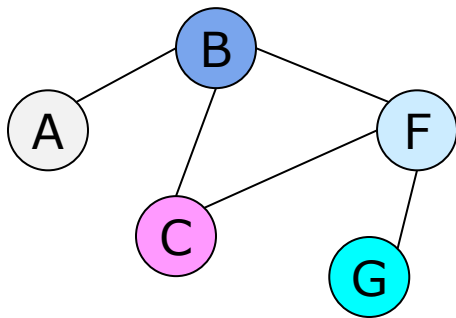
if the status of n is “not yet visited” **then** **visit**(n)

end;

□ **Breadth-first search**
with **queue**-based visits

3.3. Graph traversal algorithms

□ **Breadth-first search** with **queue**-based visits

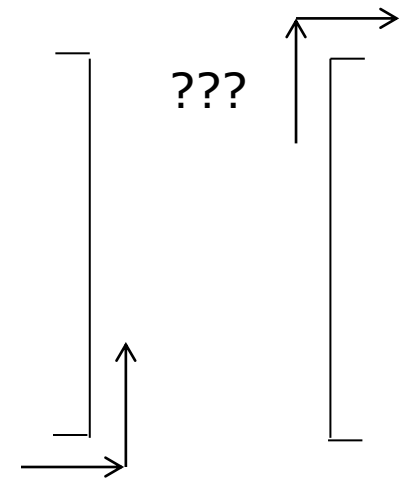


Graph

Starting at **F**: ???

A: -B
B: -A-C-F
C: -B-F
D: -I
F: -B-C-G
G: -F
I: -D

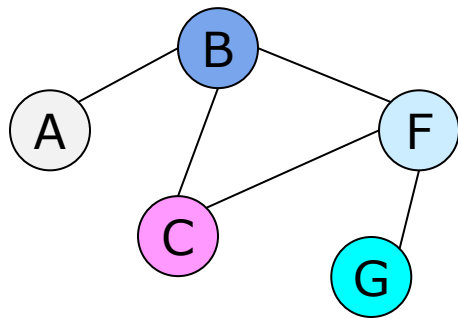
Adjacency lists



queue

3.3. Graph traversal algorithms

□ **Breadth-first search** with **queue**-based visits



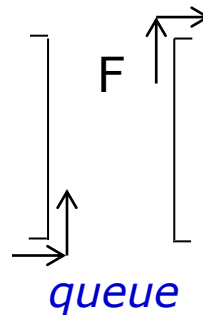
Graph

A: -B
B: -A-C-F
C: -B-F
D: -I
F: -B-C-G
G: -F
I: -D

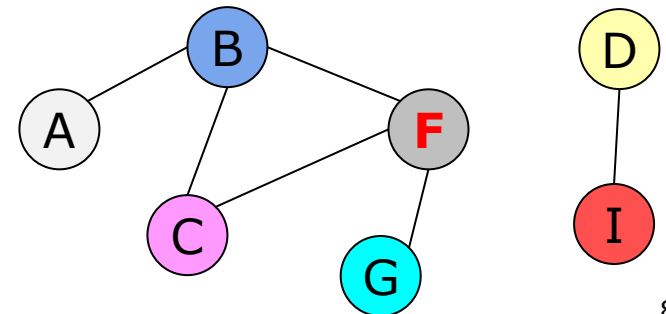
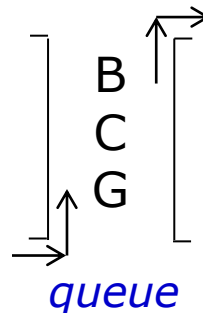
Adjacency lists

Starting at **F**:

Enqueue F

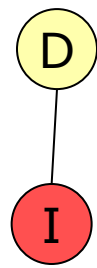
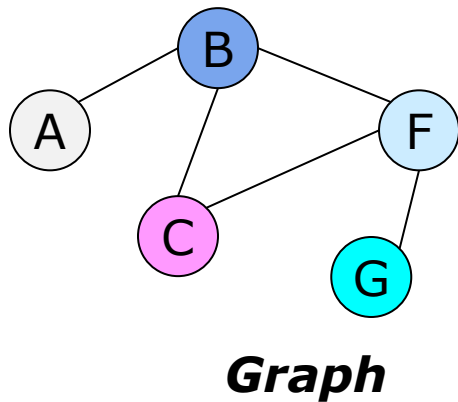


Visit F, Check F's list, Enqueue B, C, G

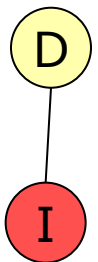
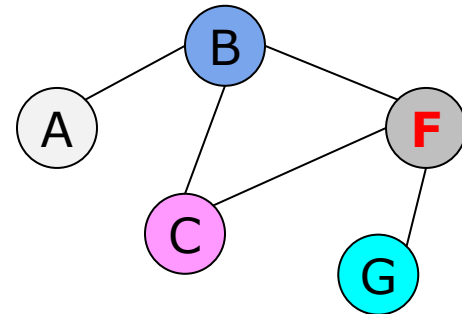
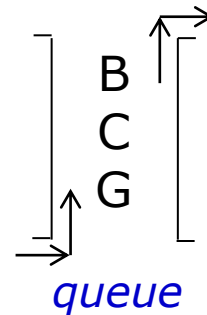


3.3. Graph traversal algorithms

□ **Breadth-first search** with **queue**-based visits



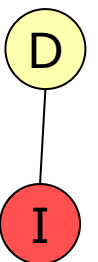
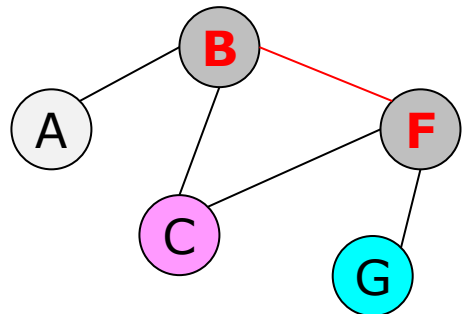
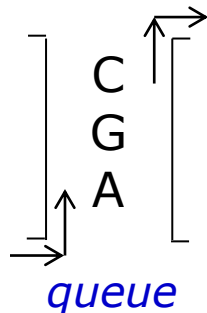
Visit F, Check F's list, Enqueue B, C, G



A: -B
B: -A-C-F
C: -B-F
D: -I
F: -B-C-G
G: -F
I: -D

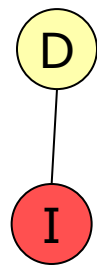
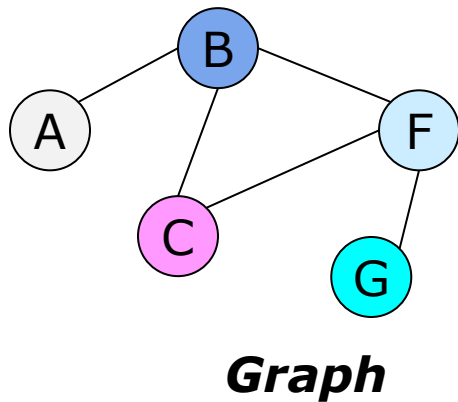
Adjacency lists

Visit B, Check B's list, Enqueue A

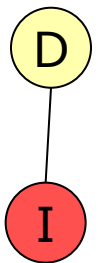
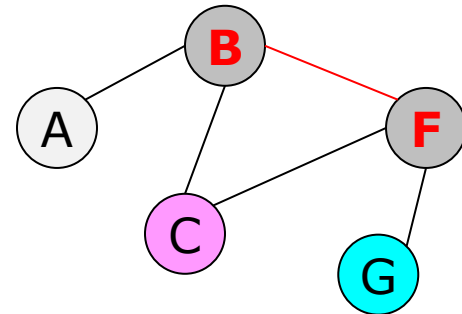
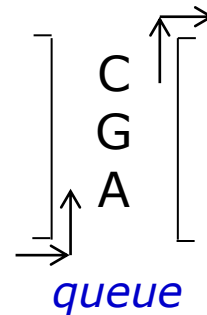


3.3. Graph traversal algorithms

□ **Breadth-first search** with **queue**-based visits



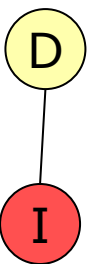
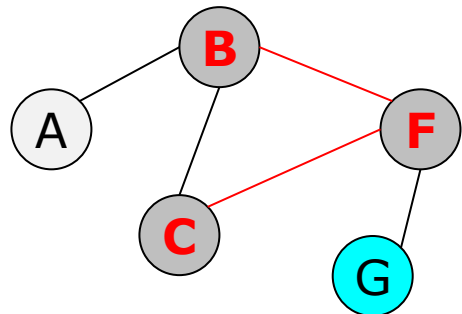
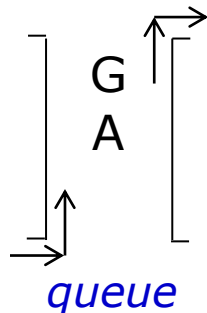
Visit B, Check B's list, Enqueue A



A: -B
B: -A-C-F
C: -B-F
D: -I
F: -B-C-G
G: -F
I: -D

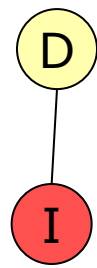
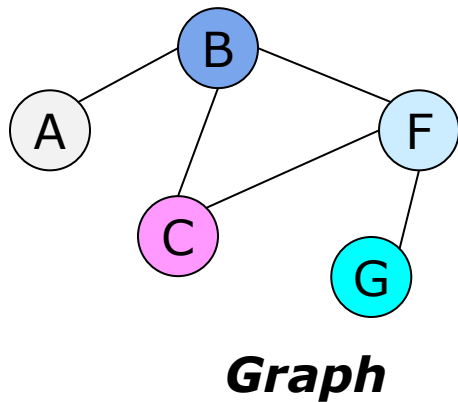
Adjacency lists

Visit C, Check C's list

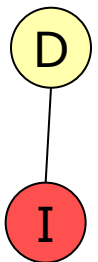
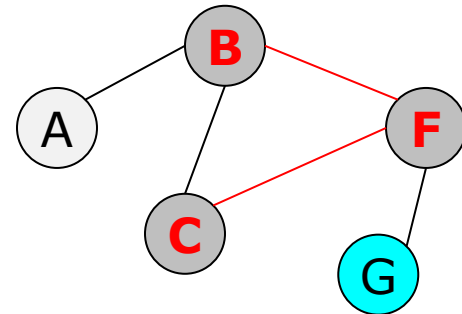
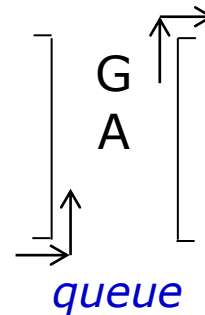


3.3. Graph traversal algorithms

□ **Breadth-first search** with **queue**-based visits



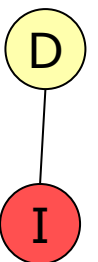
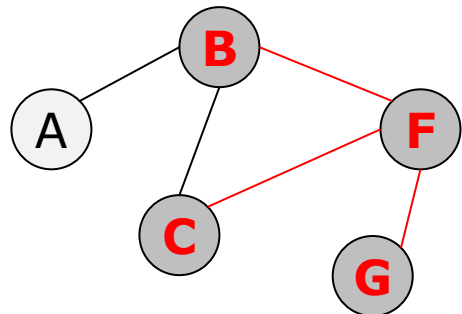
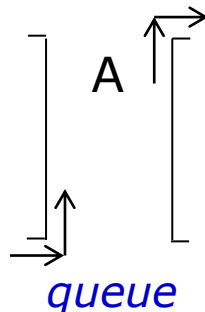
Visit C, Check C's list



A: -B
B: -A-C-F
C: -B-F
D: -I
F: -B-C-G
G: -F
I: -D

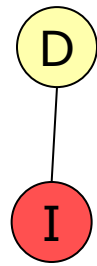
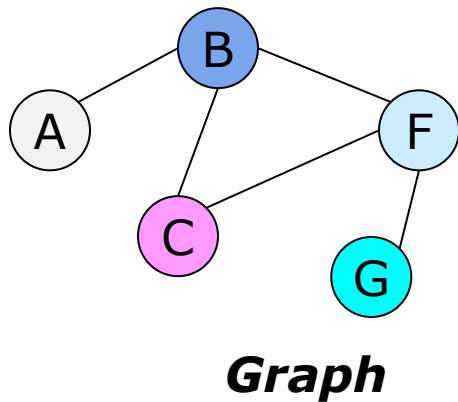
Adjacency lists

Visit G, Check G's list

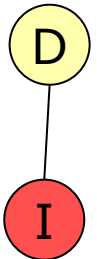
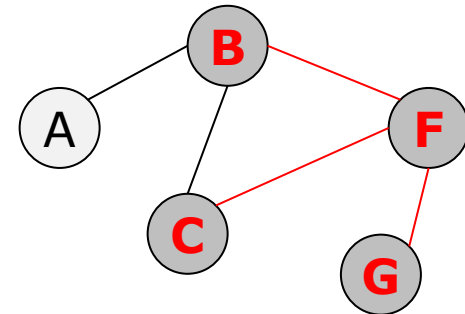
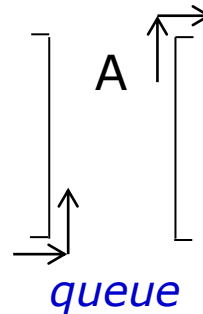


3.3. Graph traversal algorithms

□ **Breadth-first search** with **queue**-based visits



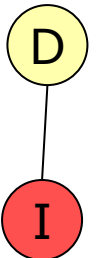
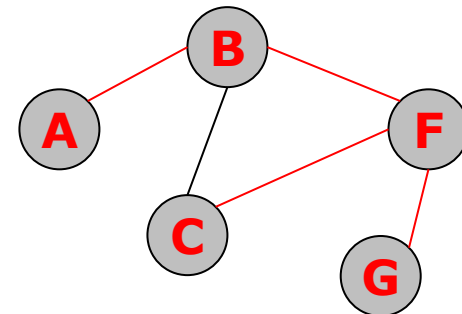
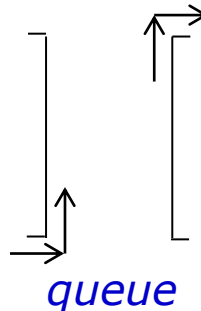
Visit G, Check G's list



A: -B
B: -A-C-F
C: -B-F
D: -I
F: -B-C-G
G: -F
I: -D

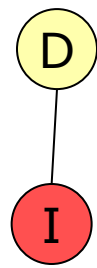
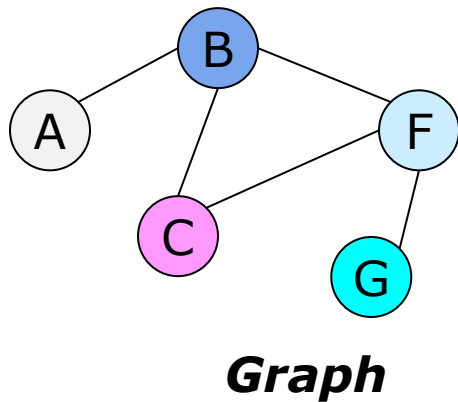
Adjacency lists

Visit A, Check A's list

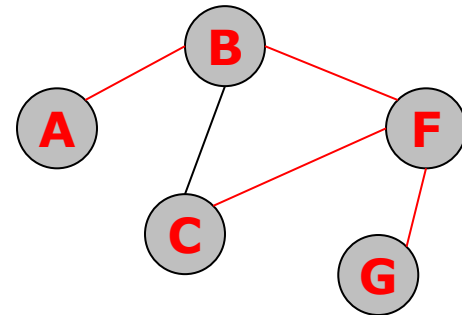
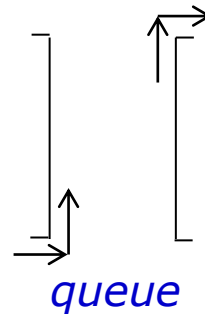


3.3. Graph traversal algorithms

□ **Breadth-first search** with **queue**-based visits



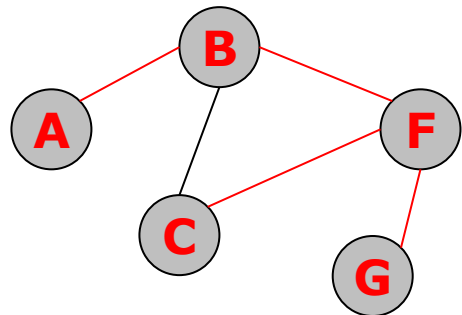
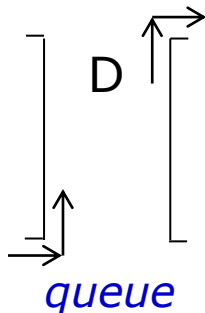
Visit A, Check A's list



A: -B
B: -A-C-F
C: -B-F
D: -I
F: -B-C-G
G: -F
I: -D

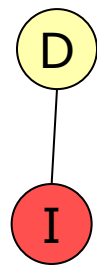
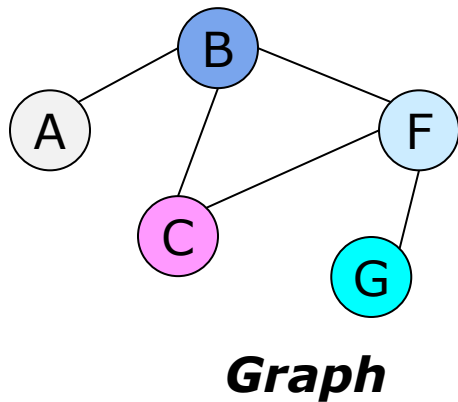
Adjacency lists

Enqueue D

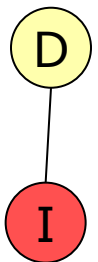
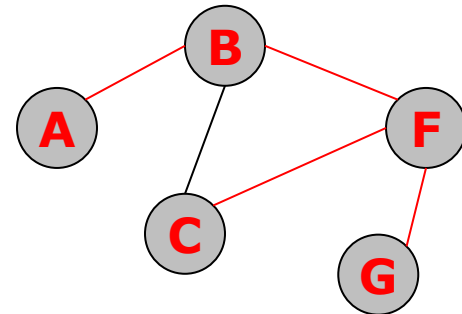
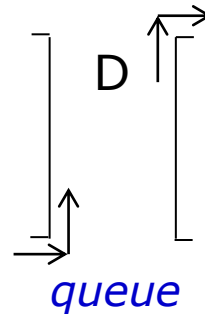


3.3. Graph traversal algorithms

□ **Breadth-first search** with **queue**-based visits



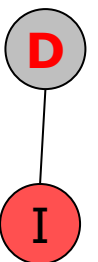
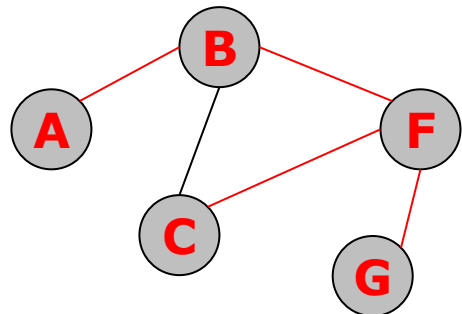
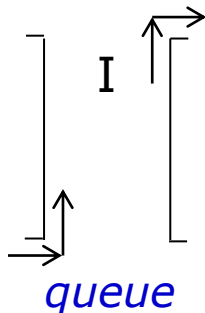
Enqueue D



A: -B
B: -A-C-F
C: -B-F
D: -I
F: -B-C-G
G: -F
I: -D

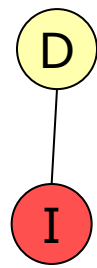
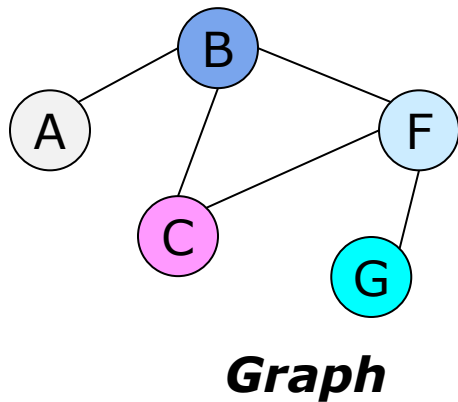
Adjacency lists

Visit D, Check D's list, Enqueue I

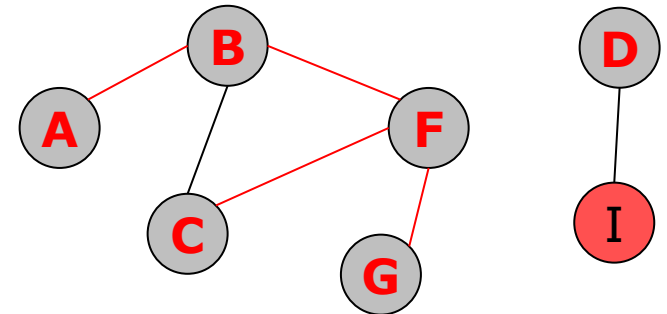
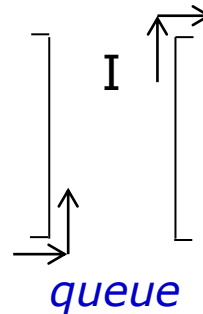


3.3. Graph traversal algorithms

□ **Breadth-first search** with **queue**-based visits



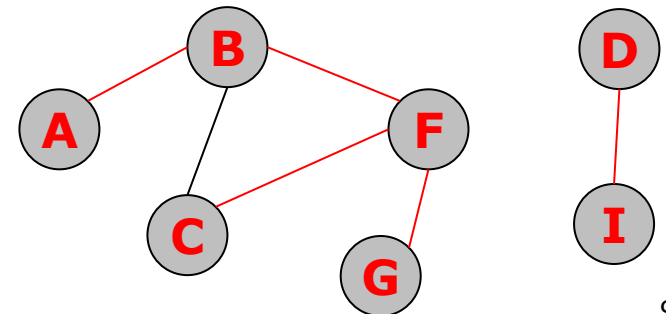
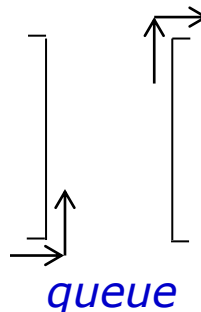
Visit D, Check D's list, Enqueue I



A: -B
B: -A-C-F
C: -B-F
D: -I
F: -B-C-G
G: -F
I: -D

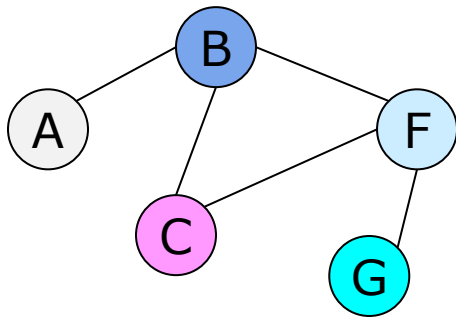
Adjacency lists

Visit I, Check I's list



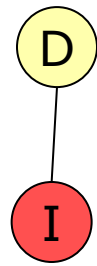
3.3. Graph traversal algorithms

□ **Breadth-first search** with **queue**-based visits



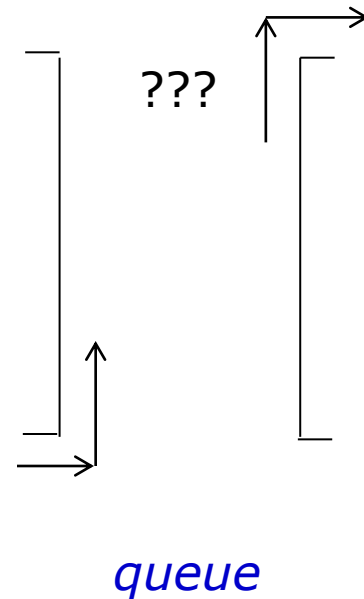
Graph

A: -B
B: -A-C-F
C: -B-F
D: -I
F: -B-C-G
G: -F
I: -D



Starting at **F**:

- Visit **F**, Enqueue B, C, G
- Visit **B**, Enqueue A
- Visit **C**
- Visit **G**
- Visit **A**
- Visit **D**, Enqueue I
- Visit **I**





3.3. Graph traversal algorithms

- Time complexity of graph traversal on a graph of V vertices and E edges
 - represented by adjacency lists
 - $O(|E| + |V|)$
 - represented by an adjacency matrix
 - $O(|V|^2)$



3.3. Graph traversal algorithms

- Time complexity of graph traversal on a graph of V vertices and E edges
 - represented by adjacency lists
 - $O(|E| + |V|)$
 - *Initialization*: $O(|V|)$
 - *Visit*:
All V vertices are processed: $O(|V|)$
One vertex is linked to its neighbors.
⇒ All E edges are accessed via adjacency lists for processing.
⇒ $O(|E|)$



3.3. Graph traversal algorithms

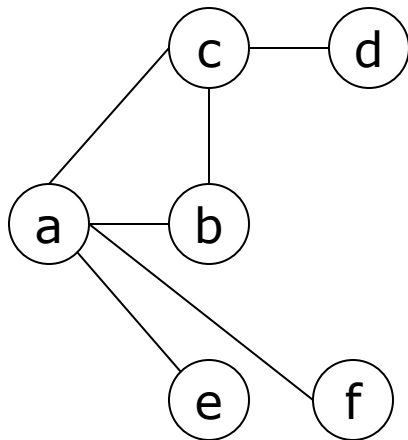
- Time complexity of graph traversal on a graph of V vertices and E edges
 - represented by an adjacency matrix
 - $O(|V|^2)$
 - *Initialization*: $O(|V|)$
 - *Visit*:
All V vertices are processed: $O(|V|)$
One vertex is linked to its neighbors.
⇒ All E edges are accessed via V -by- V matrix for processing.
⇒ $O(|V|^2)$

3.3. Graph traversal algorithms

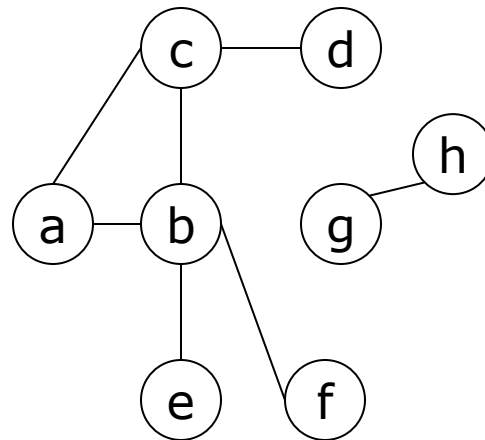
□ Your turn:

3.3.1. Traverse each following graph with vertex ***a*** as a starting vertex

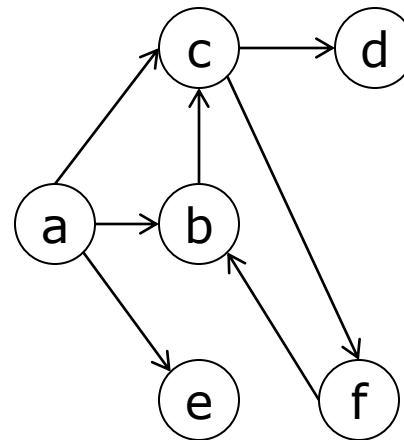
- 3.3.1.1. Depth-First Search (DFS) using stack
- 3.3.1.2. Depth-First Search (DFS) using recursion
- 3.3.1.3. Breadth-First Search (BFS) using queue



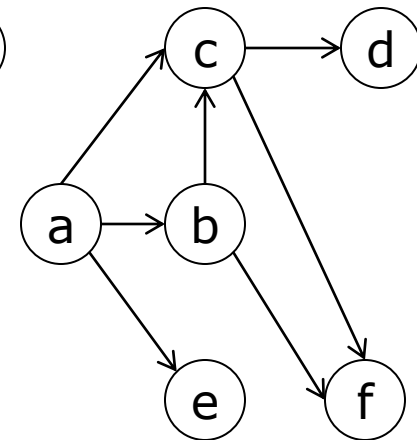
(G1)



(G2)



(G3)



(G4)

3.4. Topological sorting

- Given a *directed acyclic* graph G , a *topological sort* T of G is a linear ordering that respects the partial ordering embedded in the vertex set V of the graph.
- That means: if $u < v$ in V (i.e. if there exists a path from u to v in G), then u appears before v in the linear ordering T .

before $\xrightarrow{\dots u \dots v \dots}$ after

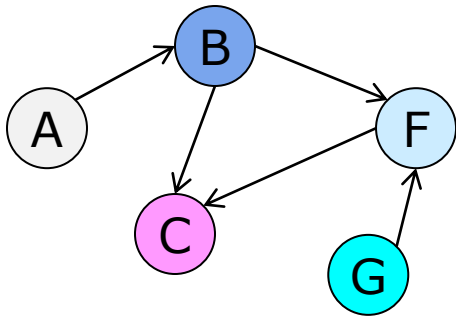
Linear ordering T

u : predecessor of v
 v : successor of u

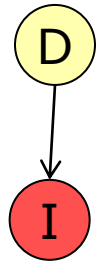
Applications of topological sorting:

- Find a sequence of courses for prerequisite requirements
- Model an assembly line
- ... and many ... with precedence relationships in domains

3.4. Topological sorting



Graph



Topological sorting T:

A, B, G, F, C, D, I

before $\longrightarrow$ after

A: -B
B: -C-F
C: -
D: -I
F: -C
G: -F
I: -

Adjacency lists

Find a topological sorting in a graph:

- DFS-based method
- Source removal method



3.4. Topological sorting

□ DFS-based method

Start with vertices with no predecessor, put them in the stack.

while the stack is not empty **do**

if the vertex at top of the stack has some successors

then

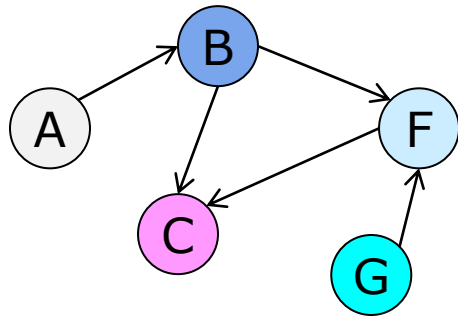
 push all its successors (not yet processed) onto the stack

else pop it from the stack, remove it from the graph and add it to the list.

Reverse the list to get the topological sorting

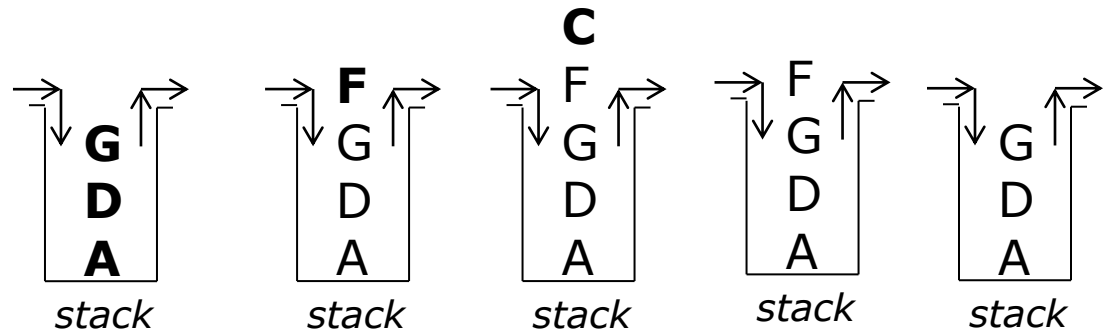
3.4. Topological sorting

DFS-based method



Graph

Starting at vertices with indegree = 0:



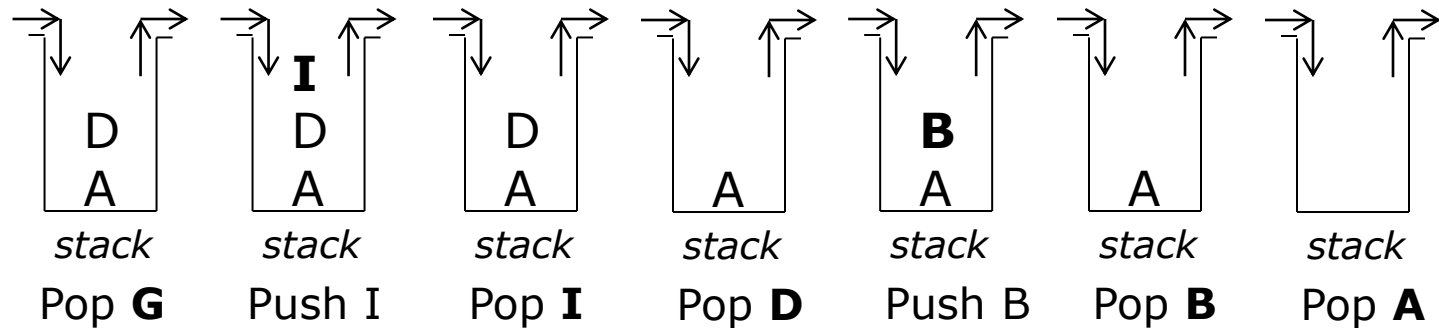
Push A, D, G

Push F

Push C

Pop **C**

Pop **F**



Pop **G**

Push I

Pop **I**

Pop **D**

Push B

Pop **B**

Pop **A**

DFS result:

C, F, G, I, D, B, A

Topological sorting:

A, B, D, I, G, F, C

Adjacency lists



3.4. Topological sorting

□ Source removal method

Start with vertices with no predecessor, put them in the queue.

while the queue is not empty **do**

 remove the front vertex N of the queue

for each neighbor vertex M of vertex N **do**

 delete the edge from N to M

if the vertex M has no predecessor **then**

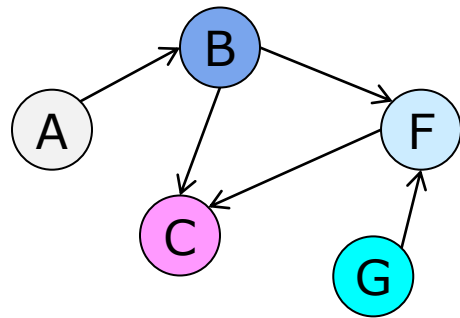
 add M to the rear of the queue

endfor

endwhile

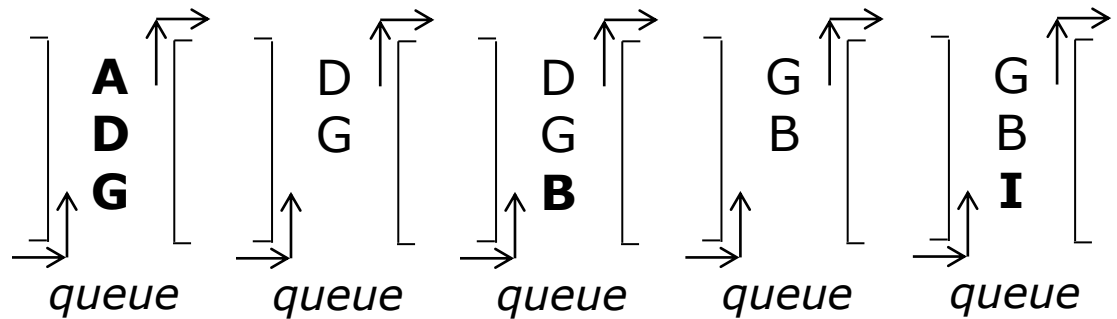
3.4. Topological sorting

□ Source removal method



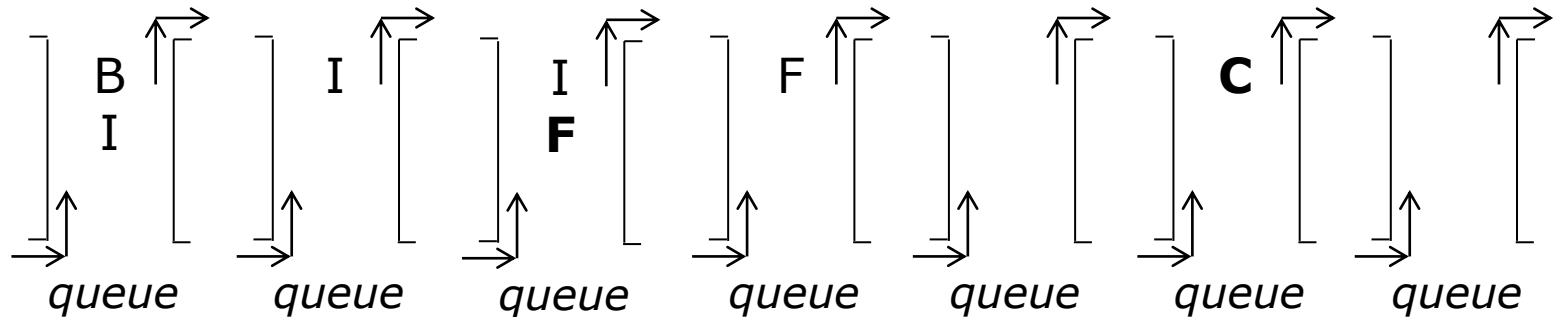
Graph

Starting at vertices with indegree = 0:



Enqueue A, D, G Dequeue **A** Enqueue B Dequeue **D** Enqueue I

A: -B
B: -C-F
C: -
D: -I
F: -C
G: -F
I: -



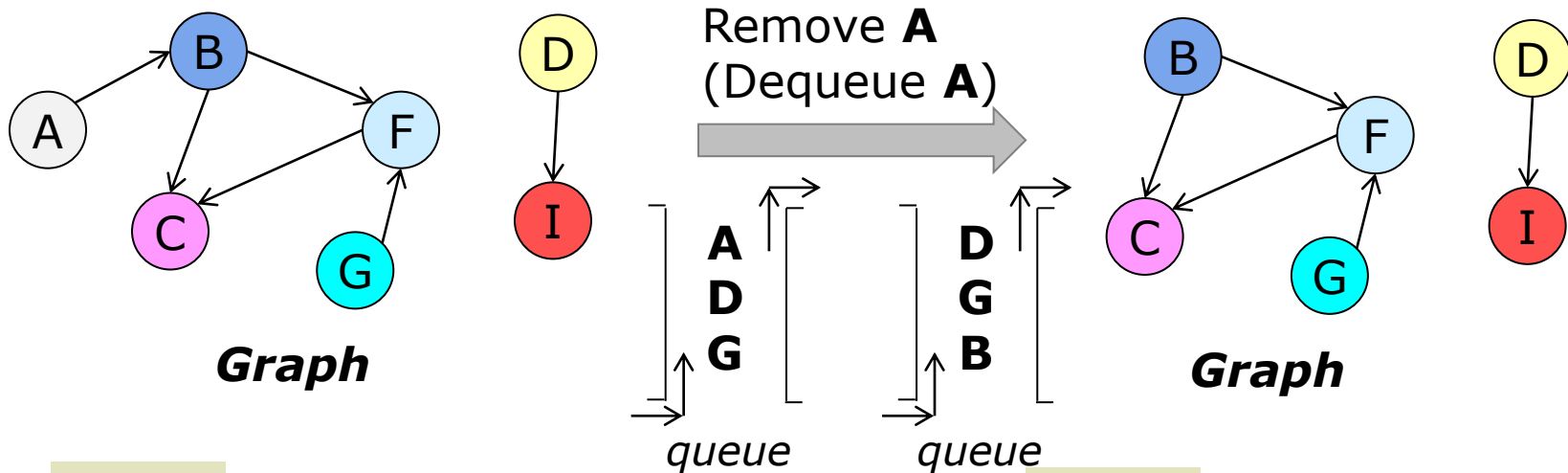
Dequeue **G** Dequeue **B** Enqueue F Dequeue **I** Dequeue **F** Enqueue C Dequeue **C**

Adjacency lists

Topological sorting: A, D, G, B, I, F, C

3.4. Topological sorting

□ Source removal method



A: -B
B: -C-F
C: -
D: -I
F: -C
G: -F
I: -

Topological sorting:

A

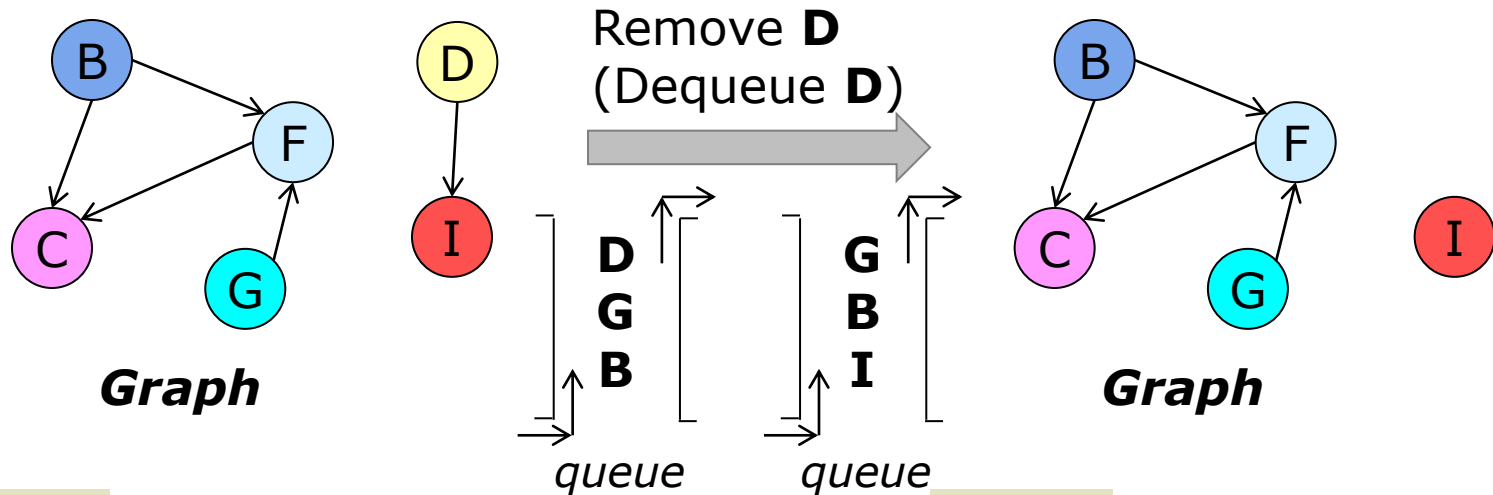
A: -B
B: -C-F
C: -
D: -I
F: -C
G: -F
I: -

Adjacency lists

Adjacency lists

3.4. Topological sorting

□ Source removal method



A: -B
B: -C-F
C: -
D: -I
F: -C
G: -F
I: -

Topological sorting:

A, D

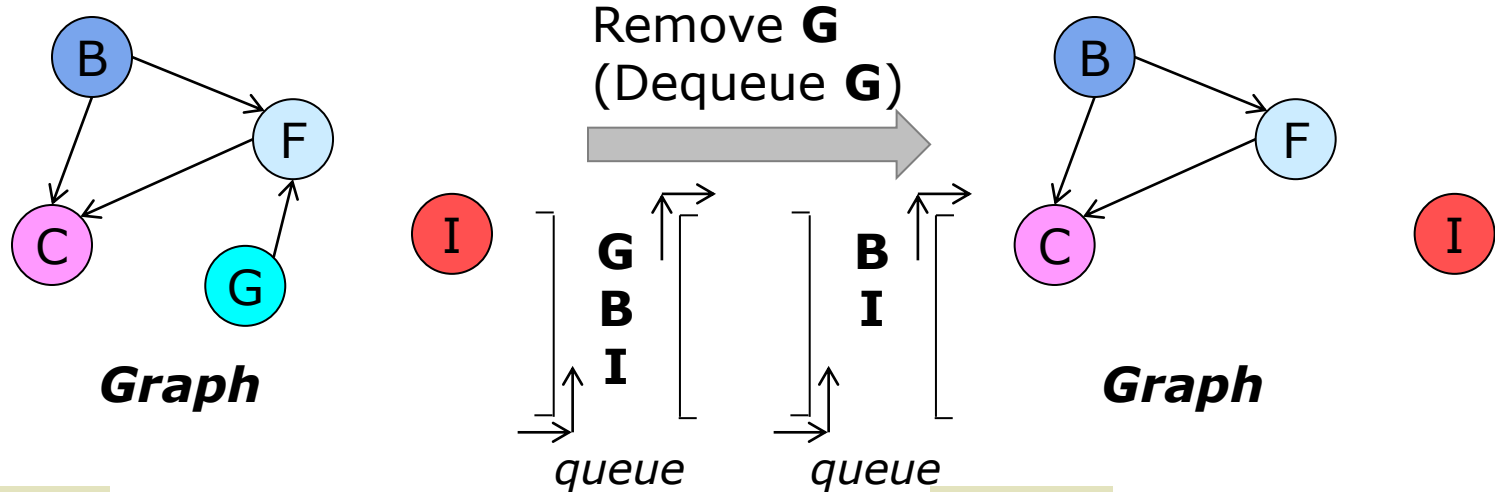
A: -B
B: -C-F
C: -
~~**D:** -I~~
F: -C
G: -F
I: -

Adjacency lists

Adjacency lists

3.4. Topological sorting

□ Source removal method



A: -B
B: -C-F
C: -
D: -I
F: -C
G: -F
I: -

Topological sorting:

A, D, G

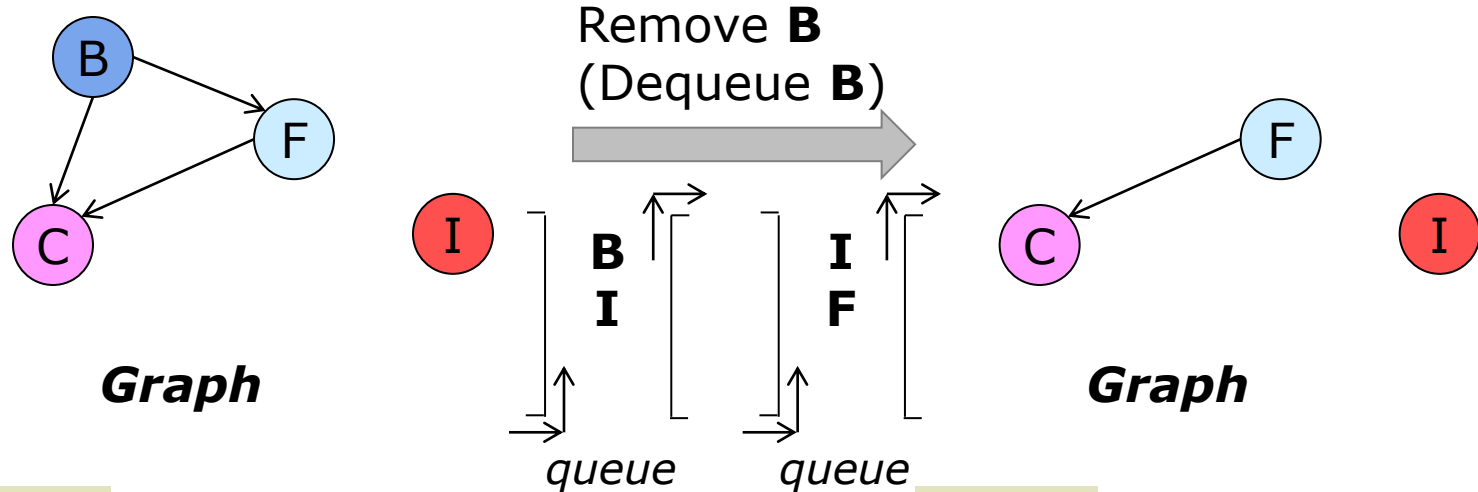
A: -B
B: -C-F
C: -
D: -I
F: -C
G: -F
I: -

Adjacency lists

Adjacency lists

3.4. Topological sorting

□ Source removal method



A: ~~B~~
B: ~~-C-F~~
C: -
D: ~~I~~
F: ~~-C~~
G: ~~F~~
I: -

Adjacency lists

Topological sorting:

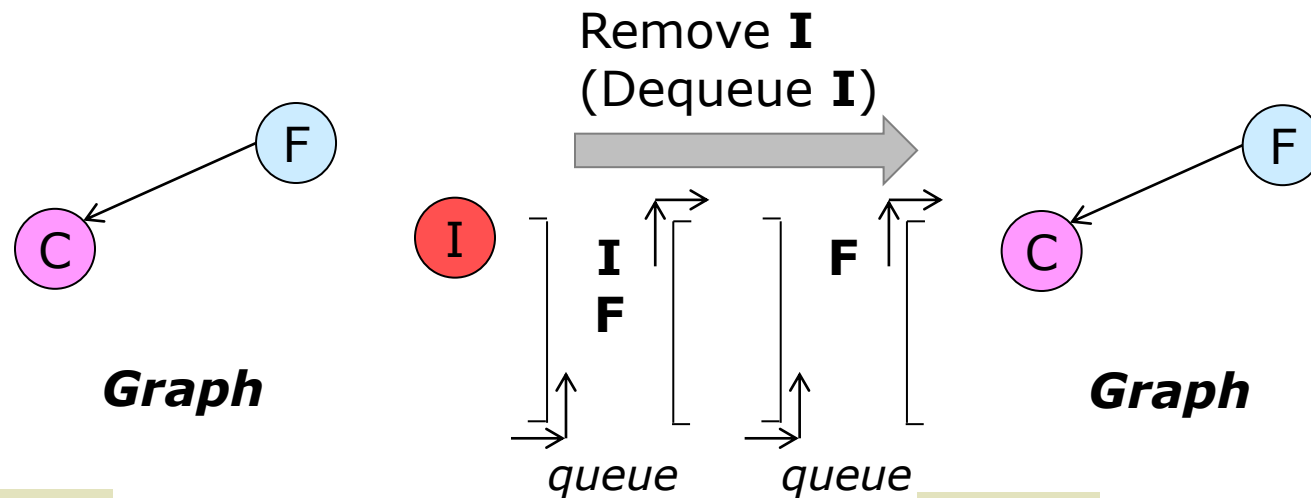
A, D, G, B

A: ~~B~~
B: ~~-C-F~~
C: -
D: ~~I~~
F: ~~-C~~
G: ~~F~~
I: -

Adjacency lists

3.4. Topological sorting

□ Source removal method



A: -B
B: -C F
C: -
D: -I
F: -C
G: -F
I: -

Topological sorting:

A, D, G, B, I

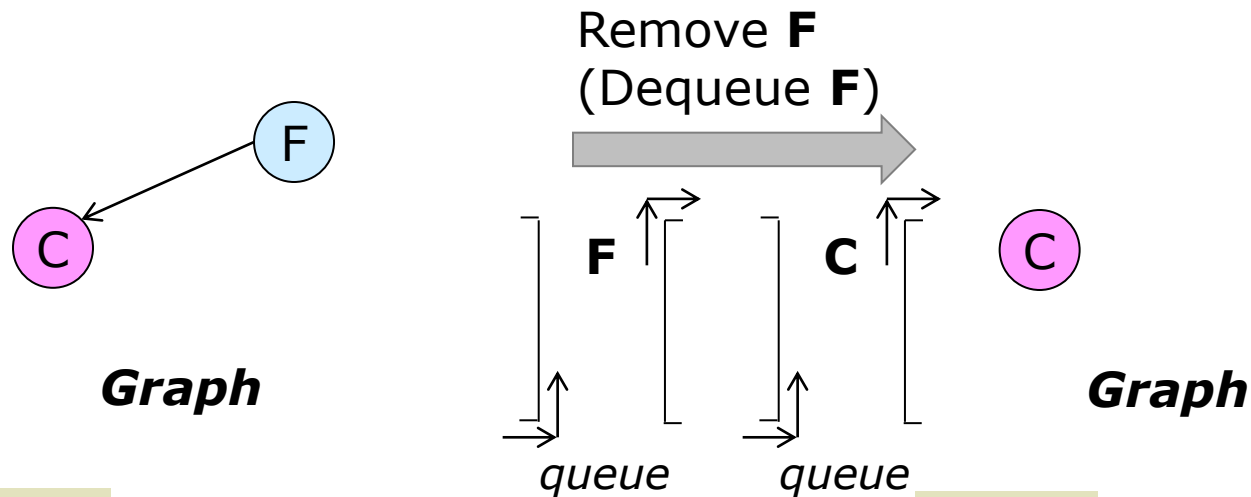
A: -B
B: -C F
C: -
D: -I
F: -C
G: -F
~~**I**~~: -

Adjacency lists

Adjacency lists

3.4. Topological sorting

□ Source removal method



A: B
B: C F
C: -
D: I
F: -C
G: F
I: -

Topological sorting:

A, D, G, B, I, F

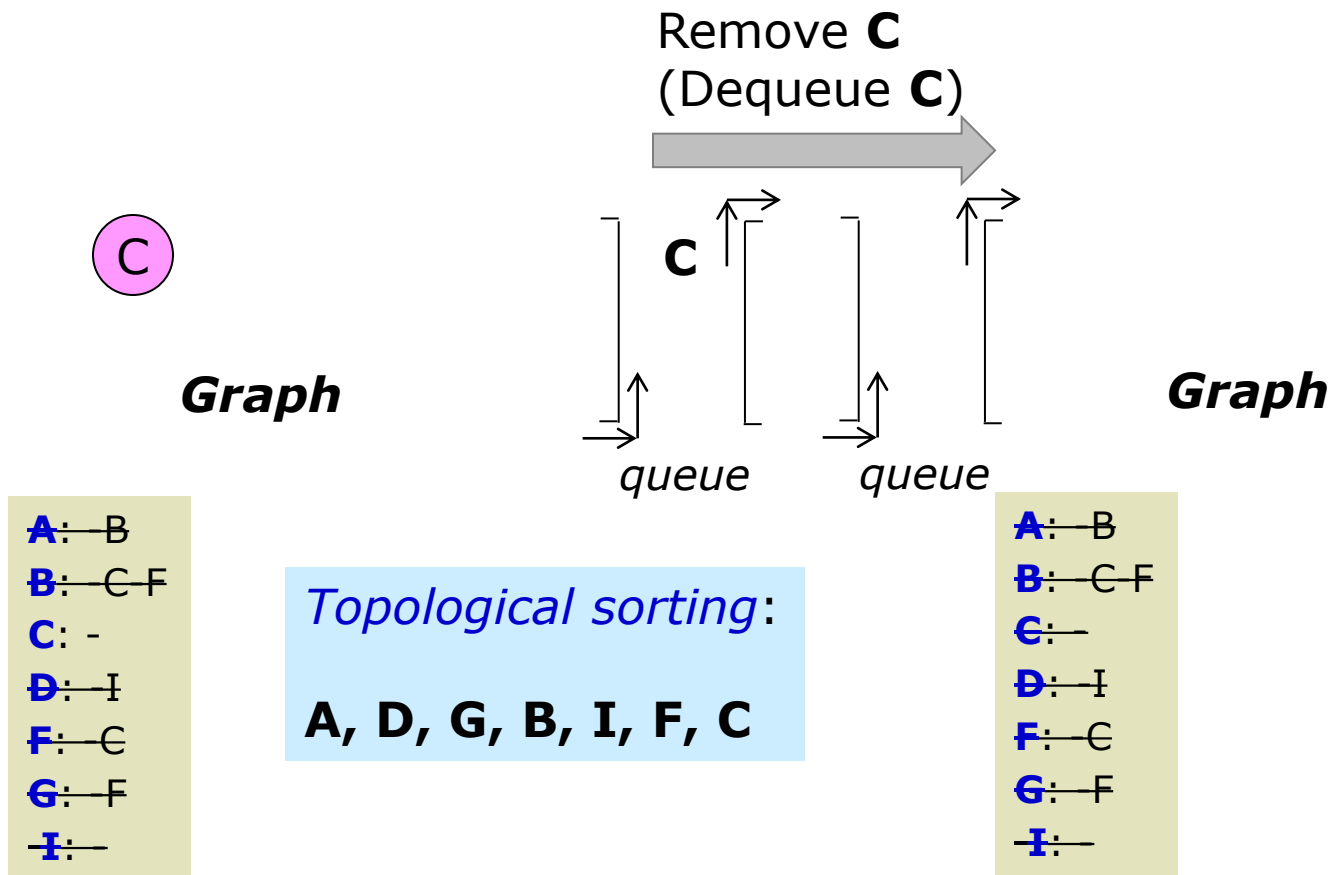
A: B
B: C F
C: -
D: I
F: C
G: F
I: -

Adjacency lists

Adjacency lists

3.4. Topological sorting

□ Source removal method





3.4. Topological sorting

- Time complexity of finding topological sorting in a directed acyclic graph of V vertices and E edges, represented by adjacency lists
 - Both DFS-based and source removal methods

$$O(|V| + |E|)$$

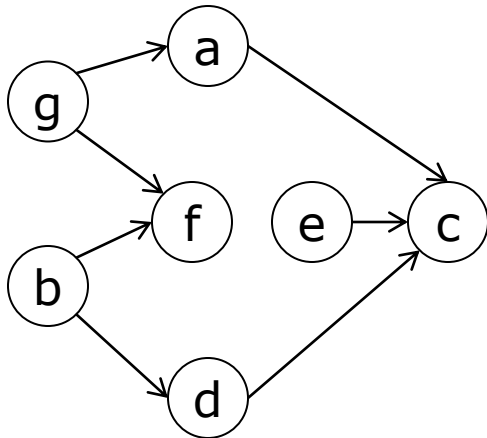
3.4. Topological sorting

□ Your turn:

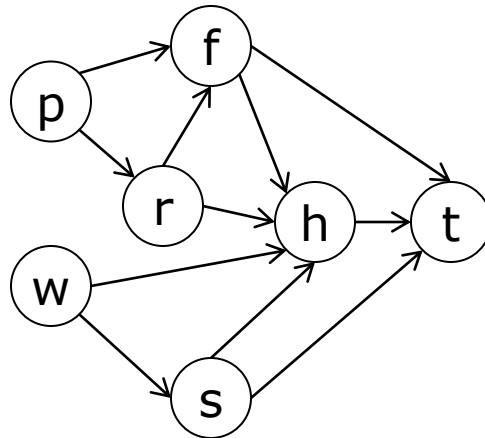
3.4.1. Find topological sorting in each graph

□ 3.4.1.1. Stack-based DFS

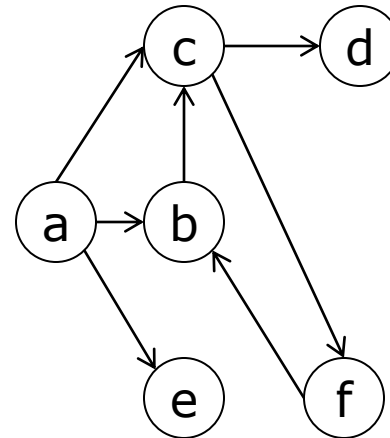
□ 3.4.1.2. Queue-based source removal



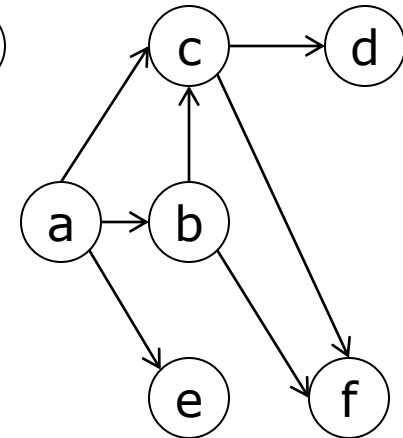
(G1)



(G2)



(G3)



(G4) 110



3.4. Topological sorting

□ Your turn:

3.4.2. Design and analyze an algorithm to find source vertices which have indegree = 0 in a graph represented by:

- 3.4.2.1. Adjacency matrix
- 3.4.2.2. Adjacency lists

3.4.3. Design and analyze an algorithm based on topological sorting to check if a graph has a cycle.



3.5. Generating all permutations from a set

- Given a set of n elements $A = \{a_1, a_2, \dots, a_n\}$, we want to generate all $n!$ different permutations from that set.
- *A smaller instance* of this problem is generating $(n-1)!$ permutations for a subset with $n-1$ elements from the set A . Assume that this smaller problem has been already solved, we can solve the original problem by *inserting the last remaining element* into every possible position in each generated permutation of the subset with $n-1$ elements.



3.5. Generating all permutations from a set

- For the convenience, assume that the set A is a set of n elements. This *permutation* set can be viewed as the set of all indices of the set of n elements $\{a_1, a_2, \dots, a_n\}$.
- An example for a set of 3 elements:

Initially 1

Insert 2 12 21

right to left

Insert 3 123 132 312 213 231 321

right to left

right to left



3.5. Generating all permutations from a set

- Time complexity for generating all permutations from a set of n elements
 - Abstract operation: *insertion of the remaining element to generate a permutation*
 - All the cases: $C_n = n! = O(n!)$

Stirling's formula

$n!$ can be approximated by the function

$$\sqrt{2\pi n} (n/e)^n$$



3.5. Generating all permutations from a set

□ Your turn:

3.5.1. Given an array A of 4 elements $\{10, 7, 9, 5\}$, generate all the permutations of the elements in A .

3.5.2. Given HeapPermute, trace by hand this algorithm for $n=2, 3, 4$. Can the algorithm generate all the permutations? What is its complexity?

ALGORITHM *HeapPermute*(n)

//Implements Heap's algorithm for generating permutations

//Input: A positive integer n and a global array $A[1..n]$

//Output: All permutations of elements of A

if $n = 1$

write A

else

for $i \leftarrow 1$ **to** n **do**

HeapPermute($n - 1$)

if n is odd

 swap $A[1]$ and $A[n]$

else swap $A[i]$ and $A[n]$

Algorithms Analysis and Design (C03031)

- Chapter 3. Decrease-and-conquer
 - 3.1. Decrease- and-conquer strategy
 - 3.2. Insertion sort
 - 3.3. Graph traversal algorithms
 - 3.4. Topological sorting
 - 3.5. Generating all permutations from a set

